CARPETA DE CAMPO PROYECTO INTERDISCIPLINARIO

GRUPO: 6

EMPRESA LOGÍSTICA

Matias Agustin Zownir

Escuela N°32 D.E N°14 Gral. José De San Martín

Computación

Profesores:Nahuel Acosta, Gabriel Abello Tuinyla

2026

24/6

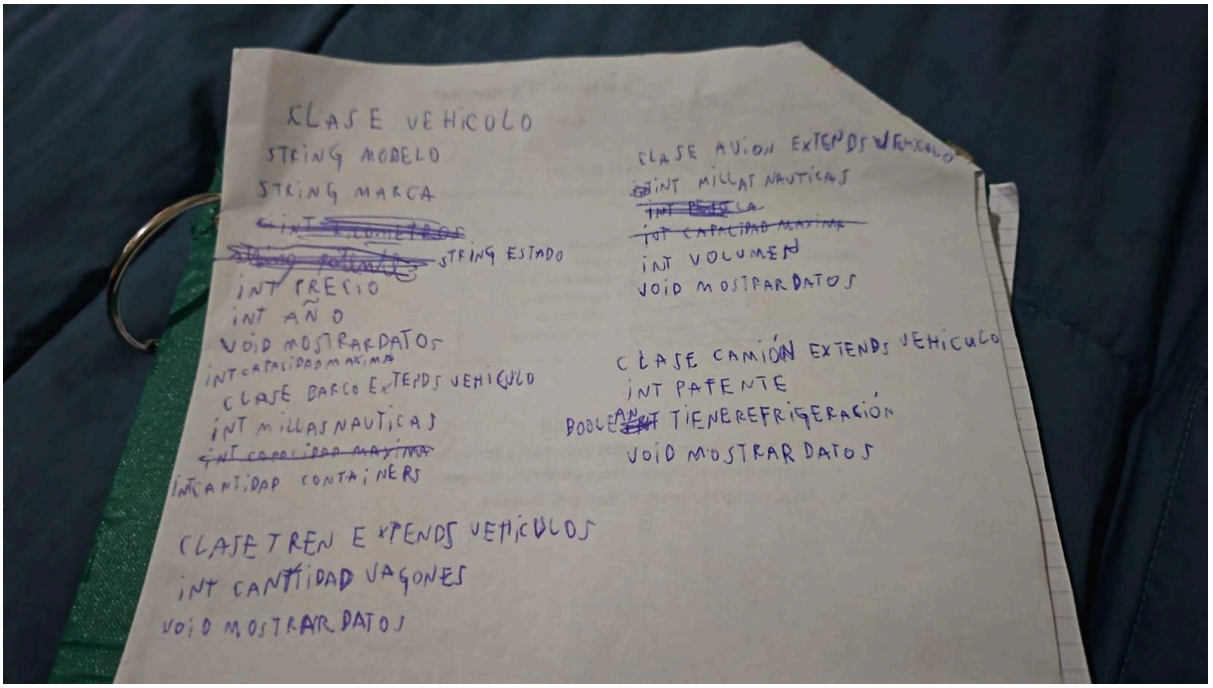
Al iniciar el proyecto, los integrantes del grupo acordamos trabajar sobre el tema de una empresa de logística. Como primera actividad, se propuso que cada uno buscara a una persona que trabajara en una empresa del rubro o que tuviera algún tipo de relación con el área, con el propósito de realizarle una entrevista y obtener información que nos permitiera conocer mejor su funcionamiento, organización y actividades principales.

En mi caso, lamentablemente, no logré encontrar a alguien que pudiera brindarme una entrevista. A pesar de no haber podido realizar esa parte de la actividad, continué trabajando junto con mis compañeros y participando en las distintas etapas del proyecto.

del 6/7 al 12/7

Nos reunimos con el grupo para proponer las distintas tablas y sus atributos, que luego serían utilizados en el desarrollo del código en Java. Una vez que definimos cómo íbamos a organizarlas, nos repartimos el trabajo y cada integrante tuvo que realizar un boceto en papel de las tablas que le habían sido asignadas.

En mi caso, me tocó diseñar la tabla Vehículo, junto con sus clases hijas: Avión, Barco, Tren y Camión. Para ello, tuve que pensar cómo organizar la información de cada tipo de vehículo y qué atributos correspondían a cada uno.



una vez hecho el boceto, esperé a que el líder del grupo santiago goldfarb revise si la tabla está bien hecha

15/07

En esta fecha, se nos asignó la tarea de pasar al código de Java las tablas que anteriormente habíamos diseñado en papel. El objetivo era comenzar a trasladar esos bocetos al proyecto y llevar su estructura a la programación.

Luego, junto con mi compañero Nicolás Rosato, realizamos algunas correcciones en el diagrama de clases UML del sistema para ajustar su organización y asegurarnos de que estuviera correctamente representado.

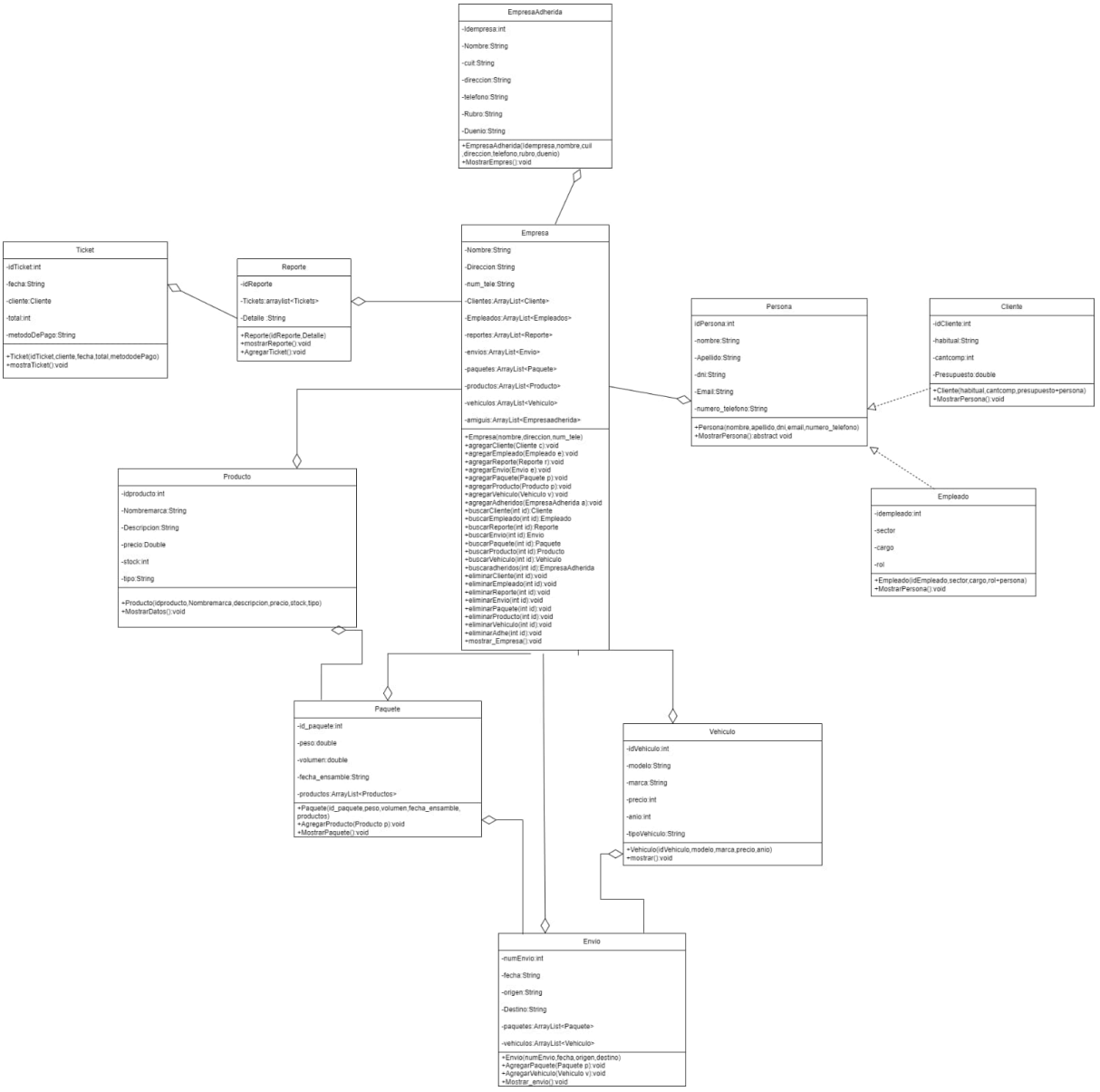
5/8

Para esta fecha, luego de finalizar el receso de invierno, se realizó la primera preentrega del proyecto. Durante esta instancia, Nahuel Acosta y Gabriel Tuinyla revisaron todo el trabajo que habíamos realizado hasta el momento. En general, el resultado fue bastante positivo, aunque recibimos algunas observaciones que debíamos tener en cuenta para continuar avanzando.

Una de las principales modificaciones fue eliminar varias tablas del diagrama UML del sistema, ya que algunas resultaban redundantes y podían generar complicaciones al momento de normalizar la base de datos.

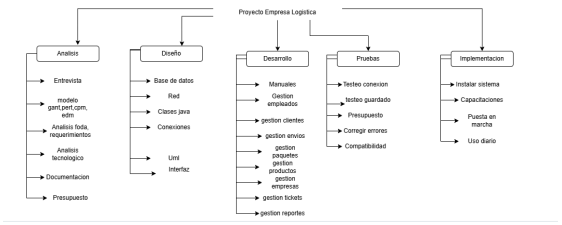
Una vez finalizada la preentrega, se me asignó, junto con mi compañero Nicolás Rosato, la tarea de rehacer el diagrama UML, quitando las tablas que ya no eran necesarias y reorganizando su estructura. De esta manera, buscamos que el diseño del sistema fuera más claro y facilitara el posterior trabajo con la base de datos.

El diagrama quedó organizado de la siguiente manera:



8/8

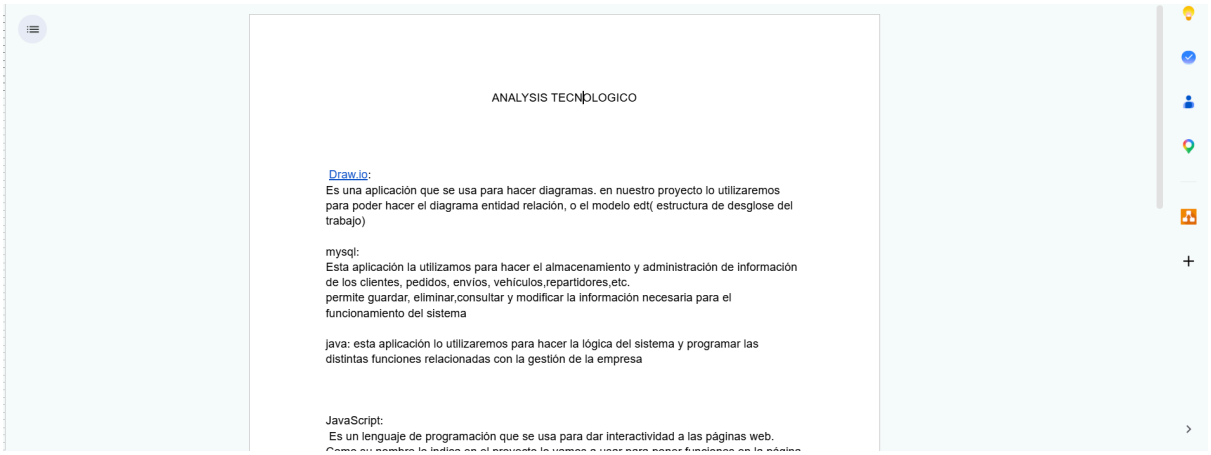
Durante esta actividad, ayudé a mi compañero Benjamin Saragusti con la elaboración del diagrama EDT del proyecto. Mi participación consistió principalmente en aportar ideas y colaborar en la identificación de los datos y elementos que podrían incluirse en el diagrama, teniendo en cuenta las distintas tareas y etapas necesarias para llevar adelante el proyecto.



9/8

realicé el trabajo que se me había asignado el viernes 7/08, que consistía en elaborar el análisis técnico del proyecto.

Para llevarlo a cabo, tuve que explicar cómo funcionaban las distintas aplicaciones que íbamos a utilizar y de qué manera se implementarían dentro de nuestro proyecto. También detallé cuál sería la utilidad de cada una y cómo nos ayudarían durante el desarrollo del sistema.



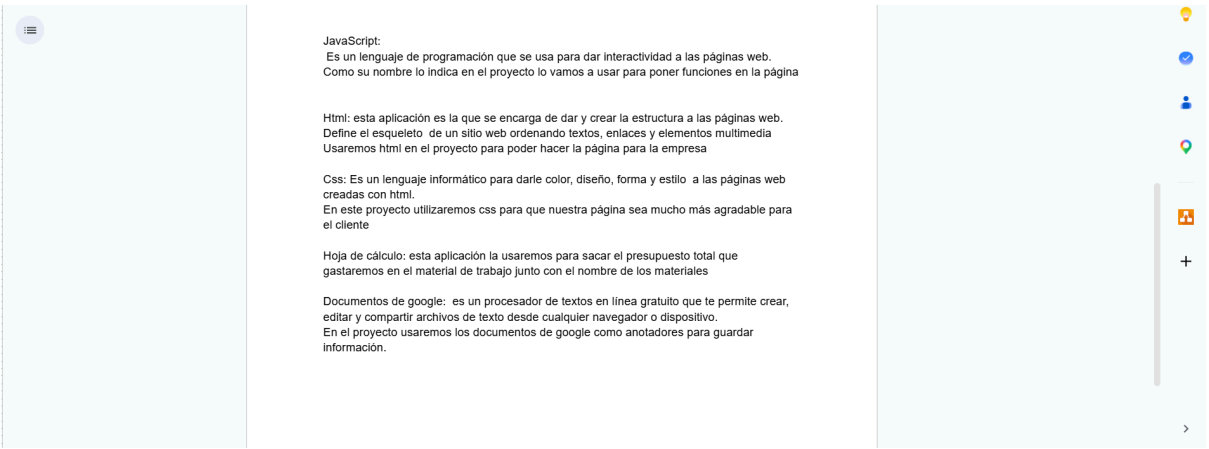
ANALYSIS TECNOLOGICO

Draw.io:
Es una aplicación que se usa para hacer diagramas. en nuestro proyecto lo utilizaremos para poder hacer el diagrama entidad relación, o el modelo edt(estructura de desglose del trabajo)

mysql:
Esta aplicación la utilizamos para hacer el almacenamiento y administración de información de los clientes, pedidos, envíos, vehículos,repartidores,etc.
permite guardar, eliminar,consultar y modificar la información necesaria para el funcionamiento del sistema

java: esta aplicación lo utilizaremos para hacer la lógica del sistema y programar las distintas funciones relacionadas con la gestión de la empresa

JavaScript:
Es un lenguaje de programación que se usa para dar interactividad a las páginas web.



JavaScript:
Es un lenguaje de programación que se usa para dar interactividad a las páginas web.
Como su nombre lo indica en el proyecto lo vamos a usar para poner funciones en la página

Html: esta aplicación es la que se encarga de dar y crear la estructura a las páginas web.
Define el esqueleto de un sitio web ordenando textos, enlaces y elementos multimedia
Usaremos html en el proyecto para poder hacer la página para la empresa

Css: Es un lenguaje informático para darle color, diseño, forma y estilo a las páginas web creadas con html.
En este proyecto utilizaremos css para que nuestra página sea mucho más agradable para el cliente

Hoja de cálculo: esta aplicación la usaremos para sacar el presupuesto total que gastaremos en el material de trabajo junto con el nombre de los materiales

Documentos de google: es un procesador de textos en línea gratuito que te permite crear, editar y compartir archivos de texto desde cualquier navegador o dispositivo.
En el proyecto usaremos los documentos de google como anotadores para guardar información.

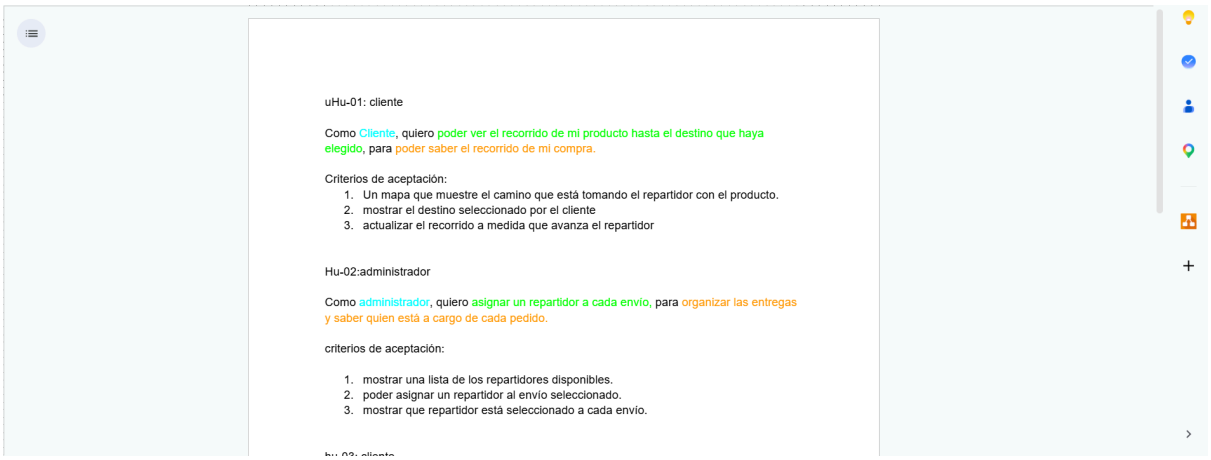
23/8

En esta fecha, me dispuse a realizar las actividades que se me habían asignado el viernes 21/08, que consistían en elaborar las historias de usuario y actualizar el DER (Diagrama Entidad-Relación) del sistema.

Para desarrollar las historias de usuario, tuve que pensar en las distintas acciones que podrían realizar tanto un administrador como un cliente interesado en nuestros servicios. Por ejemplo, del lado del cliente, consideré la posibilidad de consultar el seguimiento de su

pedido y conocer el recorrido del repartidor. Por otro lado, desde el rol de administrador, pensé en funciones como asignarle un repartidor a cada envío que se registre en el sistema.

A partir de estas ideas, busqué representar las necesidades de ambos tipos de usuarios y las funciones que debería ofrecer nuestra aplicación. Luego, también trabajé en la actualización del DER para mantener el diseño de la base de datos relacionado con los requisitos que habíamos definido.



uHu-01: cliente

Como Cliente, quiero poder ver el recorrido de mi producto hasta el destino que haya elegido, para poder saber el recorrido de mi compra.

Criterios de aceptación:

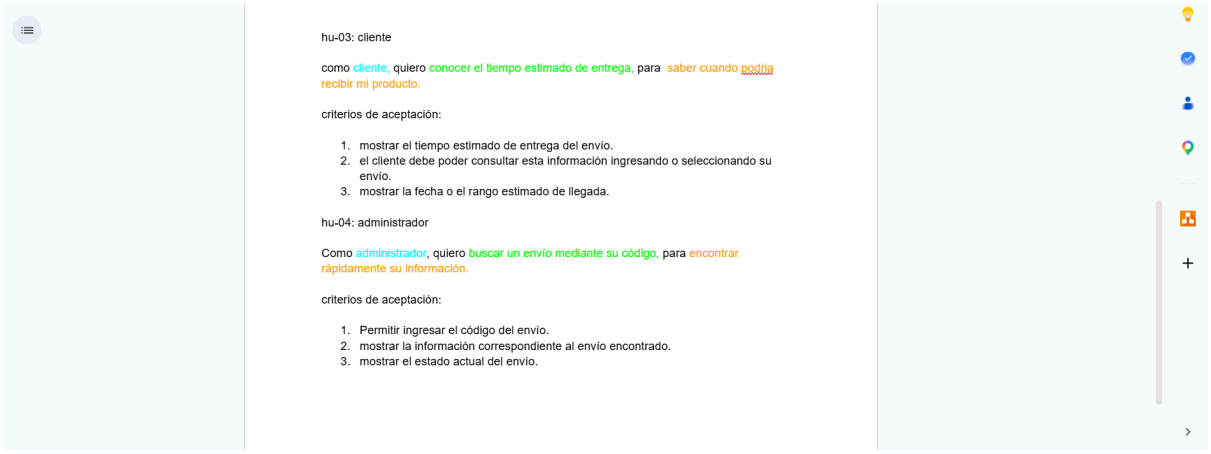
1. Un mapa que muestre el camino que está tomando el repartidor con el producto.
2. mostrar el destino seleccionado por el cliente
3. actualizar el recorrido a medida que avanza el repartidor

Hu-02:administrador

Como administrador, quiero asignar un repartidor a cada envío, para organizar las entregas y saber quien está a cargo de cada pedido.

criterios de aceptación:

1. mostrar una lista de los repartidores disponibles.
2. poder asignar un repartidor al envío seleccionado.
3. mostrar que repartidor está seleccionado a cada envío.



hu-03: cliente

como cliente, quiero conocer el tiempo estimado de entrega, para saber cuando podría recibir mi producto.

criterios de aceptación:

1. mostrar el tiempo estimado de entrega del envío.
2. el cliente debe poder consultar esta información ingresando o seleccionando su envío.
3. mostrar la fecha o el rango estimado de llegada.

hu-04: administrador

Como administrador, quiero buscar un envío mediante su código, para encontrar rápidamente su información.

criterios de aceptación:

1. Permitir ingresar el código del envío.
2. mostrar la información correspondiente al envío encontrado.
3. mostrar el estado actual del envío.

Para actualizar el DER, utilicé como referencia el diccionario de datos que ya se encontraba actualizado. A partir de él, revisé las tablas y sus atributos para identificar qué elementos era necesario agregar, modificar o eliminar. De esta manera, pude ajustar el diagrama para que coincidiera con la información que teníamos definida y mantener ambos documentos organizados.

25/8

A pedido de mi compañero Benjamín Saragusti, me encargué de realizar el conteo de horas del proyecto. Este trabajo consiste en calcular aproximadamente cuánto tiempo se necesita para completar cada una de las distintas tareas o etapas del desarrollo.

En un principio, los tiempos que estimé no fueron lo suficientemente precisos, ya que hice cálculos demasiado generales. Por este motivo, mi compañero me pidió que volviera a realizar el trabajo, esta vez revisando mejor cada actividad para establecer estimaciones más adecuadas.

Luego de rehacerlo, el conteo de horas quedó organizado de la siguiente manera:

Analisis de sistemas	cantidad horas por tarea	total
Relevamiento	4	296
Transcribir relevamiento	1	
Analisis foda	1	
Analisis requerimientos	1	
Diagrama gantt	-	
Diagrama pert	-	
Diagrama CPM Texto	2	
Diagrama CPM Dibujito	2	
Diagrama EDT	2	
Documentacion presentacion problematica y solucion	1	
Documento estado de arte	1	
MVP:Minimo producto viable	1	
Analisis tecnologico	2	
DFD	3	
Caso de uso extendido	2	
Historia de usuario	2	
Calculo de horas trabajados	1	
Manual De uso	4	
Manual de explotacion	4	
Base de datos		
Diccionario de datos	2	
Der	3	
Codificacion	6	
Tablas	1	
Relaciones	1	
Valores	6	
Funciones	15	
Vistas	2	
Procedimientos	6	

26/8

En esta etapa del proyecto, nuestro líder Goldfarb me asignó la tarea de desarrollar el CRUD junto con mi compañero Santiago Pedraza. Este trabajo consiste en crear los distintos métodos de las clases para poder realizar las operaciones necesarias sobre los datos del sistema, como agregar, consultar, modificar y eliminar registros.

Hasta el momento, logramos desarrollar dos métodos: el de agregar un cliente y el de agregar un empleado. Si bien todavía nos quedan varias operaciones por completar, ya comenzamos a construir la parte del sistema encargada de gestionar la información y esperamos poder avanzar con los métodos restantes en las próximas clases.

27/8

Con mi compañero pedraza nos dispusimos a seguir y terminar con el crud que se nos fué asignado a hacer. Terminando los demás métodos. los cuales son:
3) Agregar reporte.

Agregar envío.

Agregar paquete.

Agregar producto.

Agregar vehículo.

Agregar adheridos. (empresas adheridas)

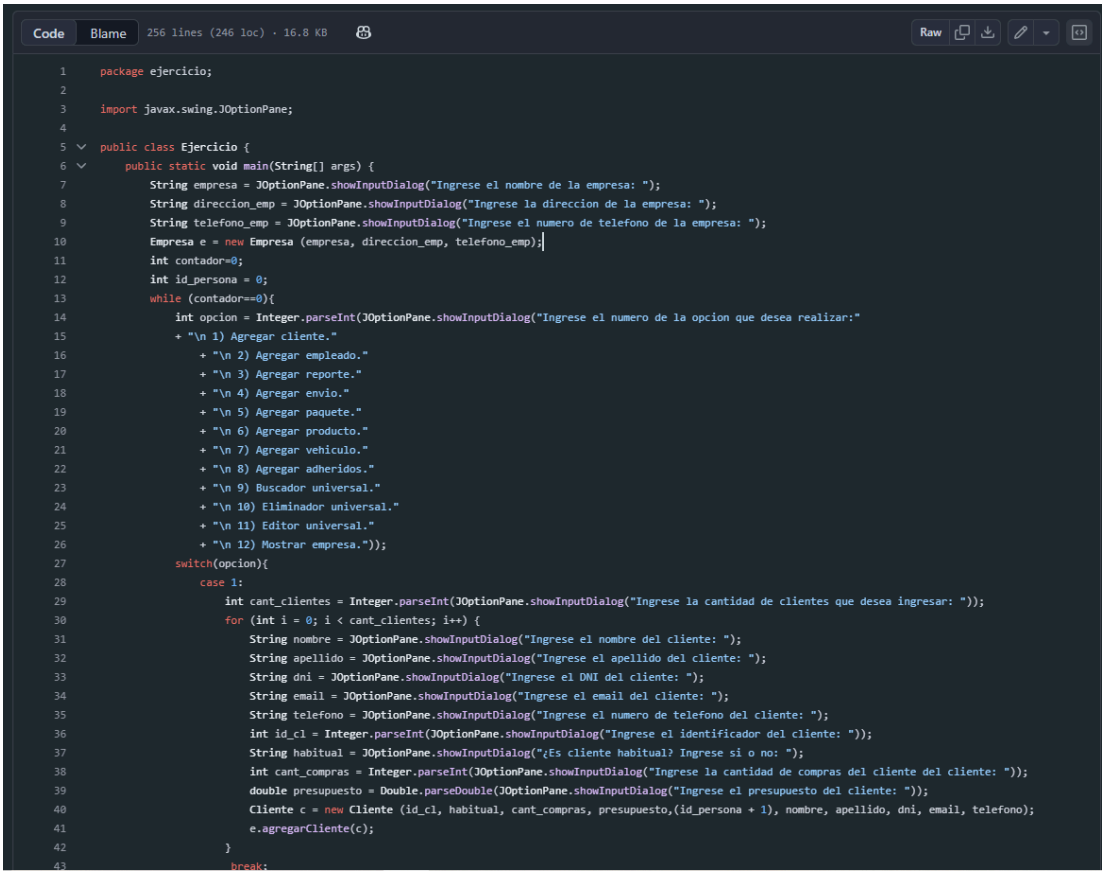
Buscador universal. (buscar cualquier elemento)

Eliminador universal. (eliminar cualquier elemento)

Editor universal. (editar cualquier elemento)

Mostrar empresa. (mostrar todos los datos y elementos de la empresa).

usamos como ayuda los Trabajos Prácticos que tenia mi compañero hechos en la materia POO, quedando tal que así:



```java
package ejercicio;

import javax.swing.JOptionPane;

public class Ejercicio {
    public static void main(String[] args) {
        String empresa = JOptionPane.showInputDialog("Ingrese el nombre de la empresa: ");
        String direccion_emp = JOptionPane.showInputDialog("Ingrese la direccion de la empresa: ");
        String telefono_emp = JOptionPane.showInputDialog("Ingrese el numero de telefono de la empresa: ");
        Empresa e = new Empresa (empresa, direccion_emp, telefono_emp);
        int contador=0;
        int id_persona = 0;
        while (contador==0){
            int opcion = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el numero de la opcion que desea realizar:"
            + "\n 1) Agregar cliente."
                + "\n 2) Agregar empleado."
                + "\n 3) Agregar reporte."
                + "\n 4) Agregar envio."
                + "\n 5) Agregar paquete."
                + "\n 6) Agregar producto."
                + "\n 7) Agregar vehiculo."
                + "\n 8) Agregar adheridos."
                + "\n 9) Buscador universal."
                + "\n 10) Eliminador universal."
                + "\n 11) Editor universal."
                + "\n 12) Mostrar empresa."));
            switch(opcion){
                case 1:
                    int cant_clientes = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de clientes que desea ingresar: "));
                    for (int i = 0; i < cant_clientes; i++) {
                        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente: ");
                        String apellido = JOptionPane.showInputDialog("Ingrese el apellido del cliente: ");
                        String dni = JOptionPane.showInputDialog("Ingrese el DNI del cliente: ");
                        String email = JOptionPane.showInputDialog("Ingrese el email del cliente: ");
                        String telefono = JOptionPane.showInputDialog("Ingrese el numero de telefono del cliente: ");
                        int id_cl = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del cliente: "));
                        String habitual = JOptionPane.showInputDialog("¿Es cliente habitual? Ingrese si o no: ");
                        int cant_compras = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de compras del cliente del cliente: "));
                        double presupuesto = Double.parseDouble(JOptionPane.showInputDialog("Ingrese el presupuesto del cliente: "));
                        Cliente c = new Cliente (id_cl, habitual, cant_compras, presupuesto,(id_persona + 1), nombre, apellido, dni, email, telefono);
                        e.agregarCliente(c);
                    }
                    break;
```

```
                    break;
                case 2:
                    int cant_empleados = Integer.parseInt(JOptionPane.showInputDialog("ingrese la cantidad de empleados que desea ingresar"));
                    for (int i = 0; i < cant_empleados; i++) {
                        String nombre = JOptionPane.showInputDialog("Ingrese el nombre del cliente: ");
                        String apellido = JOptionPane.showInputDialog("Ingrese el apellido del cliente: ");
                        String dni = JOptionPane.showInputDialog("Ingrese el DNI del cliente: ");
                        String email = JOptionPane.showInputDialog("Ingrese el email del cliente: ");
                        String telefono = JOptionPane.showInputDialog("Ingrese el numero de telefono del cliente: ");
                        int id_em =  Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del empleado: "));
                        String sector = JOptionPane.showInputDialog("ingrese el sector en donde trabaja el empleado");
                        String cargo = JOptionPane.showInputDialog("ingrese el cargo");
                        String rol = JOptionPane.showInputDialog("ingrese el rol del empleado");
                        Empleado empl = new Empleado (id_em,sector, cargo, rol,(id_persona + 1), nombre, apellido, dni, email, telefono);
                        e.agregarEmpleado(empl);
                    }
                    break;
                case 3:
                    int cant_reportes = Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de reportes que desea ingresar"));
                    for (int i = 0; i < cant_reportes; i++) {
                        int id_rep = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el identificador del reporte: "));
                        String detalle = JOptionPane.showInputDialog("ingrese el detalle del reporte: ");
                        Reporte rep = new Reporte (id_rep, detalle);
                        int cant_ticket=Integer.parseInt(JOptionPane.showInputDialog("Ingrese la cantidad de tickets a añadir :"));
                        for(int x=0;x<cant_ticket;x++){
                            int idx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el id del ticket :"));
                            String fechas=JOptionPane.showInputDialog("Ingrese la fecha del pago .");
                            int totalx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el total del pago :"));
                            String pagos=JOptionPane.showInputDialog("Ingrese el metodo de pago :");
                            int idxx=Integer.parseInt(JOptionPane.showInputDialog("Ingrese el id del cliente del ticket :"));
                            int indice=e.buscador_universal(idxx,1);
                            Cliente c=e.clientes.get(indice);
                            Ticket t=new Ticket(idx,fechas,c,totalx,pagos);
                            rep.agregarTicket(t);
                        }
                        e.agregarReporte(rep);
                    }
                    break;
```

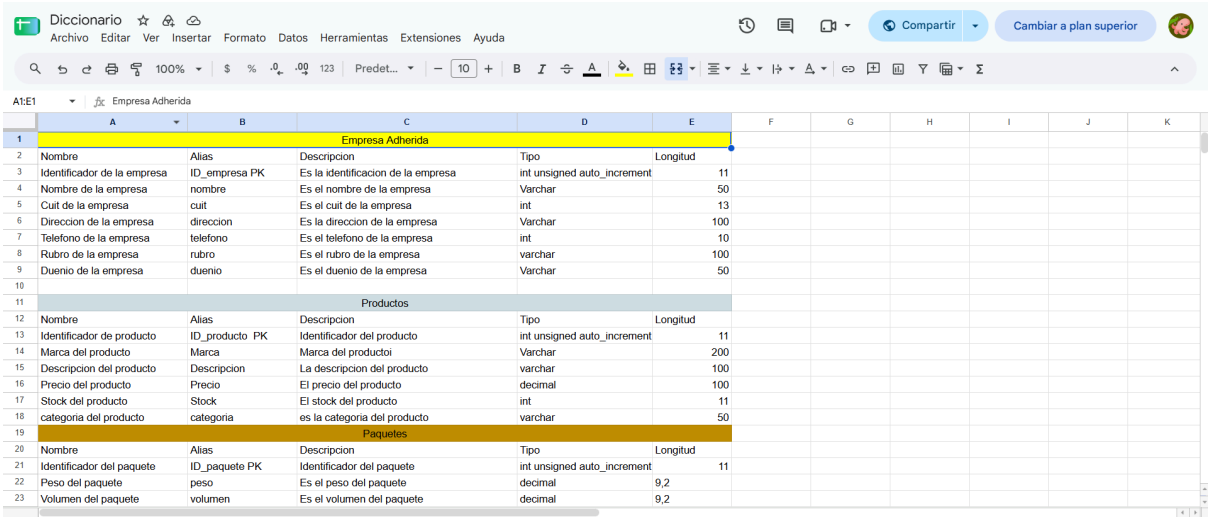

Después de la primera entrega del proyecto, los profesores nos indicaron que teníamos que realizar algunas modificaciones en la documentación de la base de datos. Una de ellas era volver a hacer el diccionario de datos, ya que el anterior no se entendía correctamente.

El diccionario de datos es un documento que sirve para describir cómo está organizada la información de una base de datos. En él se detallan las tablas que forman parte del sistema y los campos que contiene cada una, junto con sus características.

Para realizar la actualización, primero tuve que esperar a que mi compañero Santiago Pedraza terminara de rehacer el DER, ya que necesitaba que el modelo estuviera actualizado para poder trabajar sobre el diccionario.

Una vez que tuve el material, me encargué de revisar la información, quitar las clases que ya no eran necesarias y corregir algunos errores ortográficos que tenía el documento anterior.

Este trabajo fue importante porque permitió que la documentación estuviera más ordenada y fuera más fácil de comprender. Además, ayudó a mantener una relación entre el diseño de la base de datos y la información que se encontraba explicada en el diccionario.



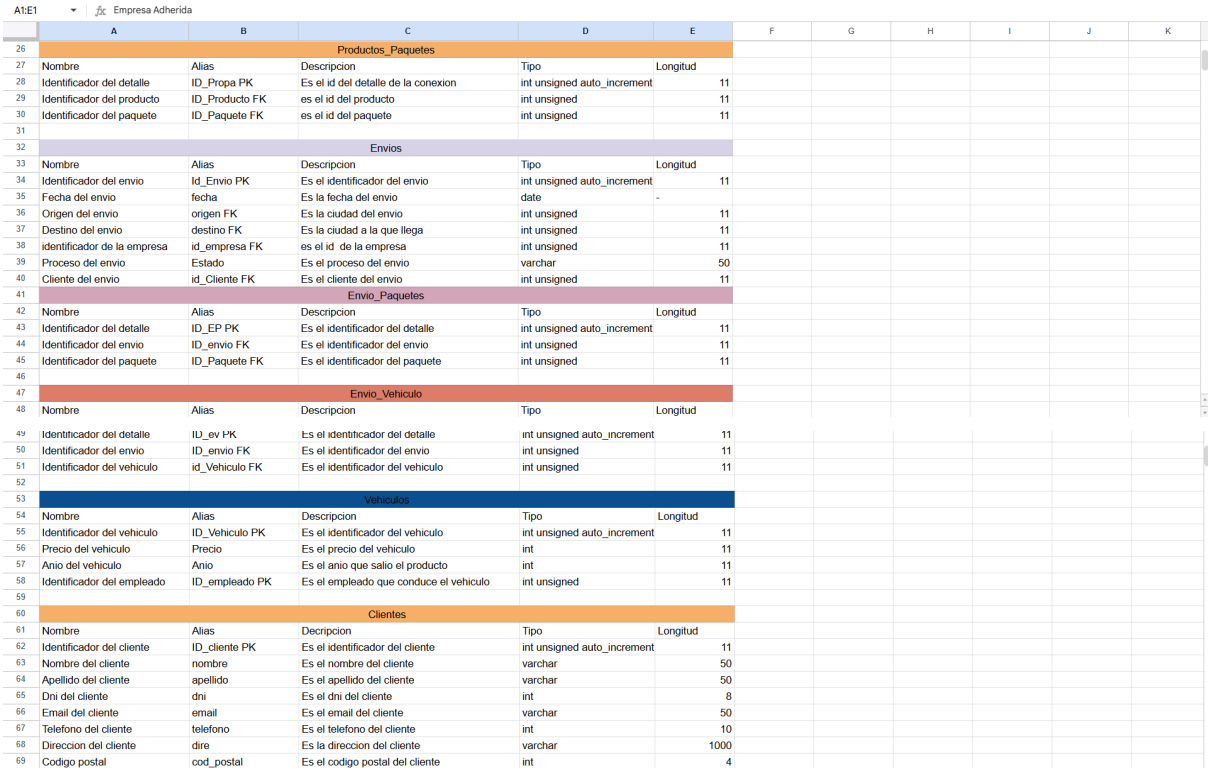
Diccionario

Archivo Editar Ver Insertar Formato Datos Herramientas Extensiones Ayuda

Compartir | Cambiar a plan superior

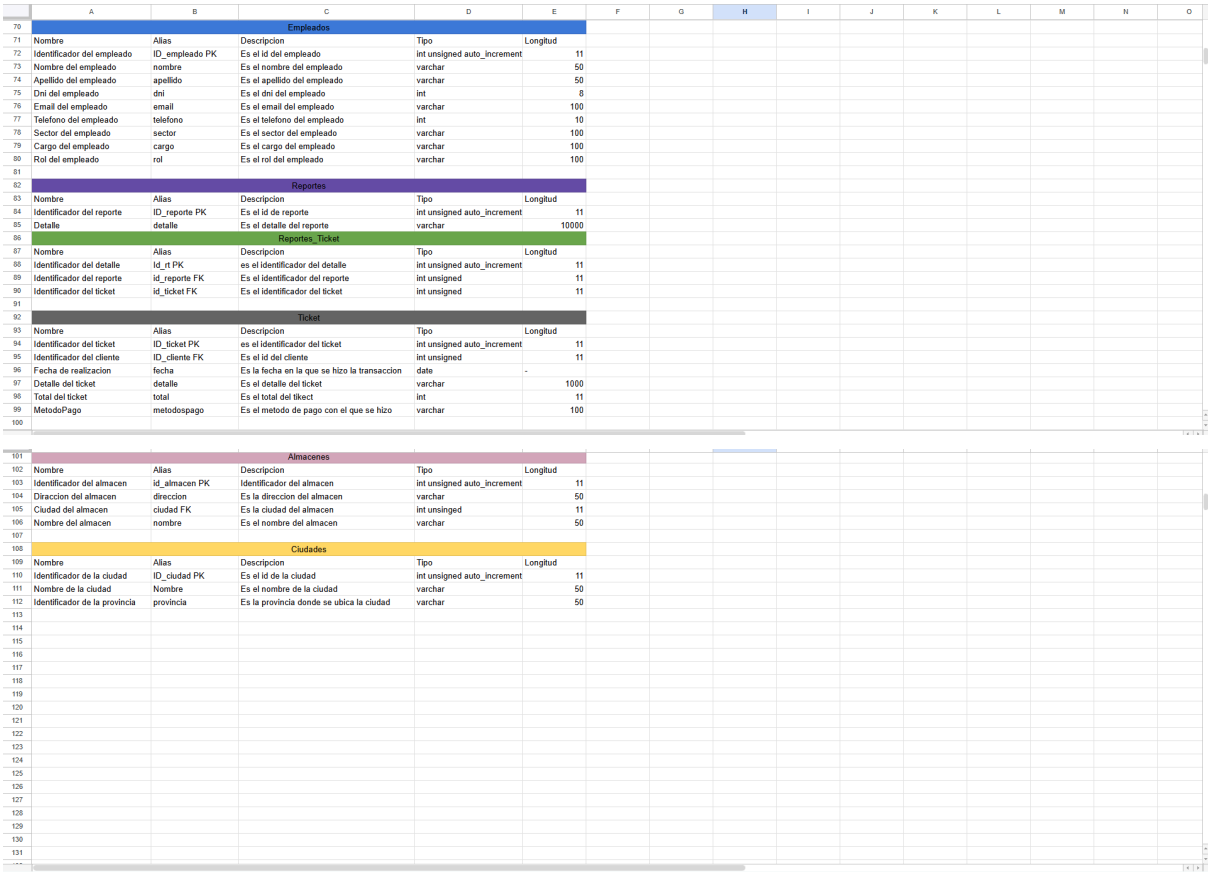
A1:E1 | Empresa Adherida

	A	B	C	D	E
1	Empresa Adherida				
2	Nombre	Alias	Descripcion	Tipo	Longitud
3	Identificador de la empresa	ID_empresa PK	Es la identificacion de la empresa	int unsigned auto_increment	11
4	Nombre de la empresa	nombre	Es el nombre de la empresa	Varchar	50
5	Cuit de la empresa	cuit	Es el cuit de la empresa	int	13
6	Direccion de la empresa	direccion	Es la direccion de la empresa	Varchar	100
7	Telefono de la empresa	telefono	Es el telefono de la empresa	int	10
8	Rubro de la empresa	rubro	Es el rubro de la empresa	varchar	100
9	Duenio de la empresa	duenio	Es el duenio de la empresa	Varchar	50
10					
11	Productos				
12	Nombre	Alias	Descripcion	Tipo	Longitud
13	Identificador de producto	ID_producto PK	Identificador del producto	int unsigned auto_increment	11
14	Marca del producto	Marca	Marca del productoi	Varchar	200
15	Descripcion del producto	Descripcion	La descripcion del producto	varchar	100
16	Precio del producto	Precio	El precio del producto	decimal	100
17	Stock del producto	Stock	El stock del producto	int	11
18	categoria del producto	categoria	es la categoria del producto	varchar	50
19	Paquetes				
20	Nombre	Alias	Descripcion	Tipo	Longitud
21	Identificador del paquete	ID_paquete PK	Identificador del paquete	int unsigned auto_increment	11
22	Peso del paquete	peso	Es el peso del paquete	decimal	9,2
23	Volumen del paquete	volumen	Es el volumen del paquete	decimal	9,2



A1:E1 | Empresa Adherida

	A	B	C	D	E
26	Productos_Paquetes				
27	Nombre	Alias	Descripcion	Tipo	Longitud
28	Identificador del detalle	ID_Propa PK	Es el id del detalle de la conexion	int unsigned auto_increment	11
29	Identificador del producto	ID_Producto FK	es el id del producto	int unsigned	11
30	Identificador del paquete	ID_Paquete FK	es el id del paquete	int unsigned	11
31					
32	Envios				
33	Nombre	Alias	Descripcion	Tipo	Longitud
34	Identificador del envio	Id_Envio PK	Es el identificador del envio	int unsigned auto_increment	11
35	Fecha del envio	fecha	Es la fecha del envio	date	-
36	Origen del envio	origen FK	Es la ciudad del envio	int unsigned	11
37	Destino del envio	destino FK	Es la ciudad a la que llega	int unsigned	11
38	identificador de la empresa	id_empresa FK	es el id de la empresa	int unsigned	11
39	Proceso del envio	Estado	Es el proceso del envio	varchar	50
40	Cliente del envio	id_Cliente FK	Es el cliente del envio	int unsigned	11
41	Envio_Paquetes				
42	Nombre	Alias	Descripcion	Tipo	Longitud
43	Identificador del detalle	ID_EP PK	Es el identificador del detalle	int unsigned auto_increment	11
44	Identificador del envio	ID_envio FK	Es el identificador del envio	int unsigned	11
45	Identificador del paquete	ID_Paquete FK	Es el identificador del paquete	int unsigned	11
46					
47	Envio_Vehiculo				
48	Nombre	Alias	Descripcion	Tipo	Longitud
49	Identificador del detalle	ID_ev PK	Es el identificador del detalle	int unsigned auto_increment	11
50	Identificador del envio	ID_envio FK	Es el identificador del envio	int unsigned	11
51	Identificador del vehiculo	id_Vehiculo FK	Es el identificador del vehiculo	int unsigned	11
52					
53	Vehiculos				
54	Nombre	Alias	Descripcion	Tipo	Longitud
55	Identificador del vehiculo	ID_Vehiculo PK	Es el identificador del vehiculo	int unsigned auto_increment	11
56	Precio del vehiculo	Precio	Es el precio del vehiculo	int	11
57	Anio del vehiculo	Anio	Es el anio que salio el producto	int	11
58	Identificador del empleado	ID_empleado PK	Es el empleado que conduce el vehiculo	int unsigned	11
59					
60	Clientes				
61	Nombre	Alias	Decripcion	Tipo	Longitud
62	Identificador del cliente	ID_cliente PK	Es el identificador del cliente	int unsigned auto_increment	11
63	Nombre del cliente	nombre	Es el nombre del cliente	varchar	50
64	Apellido del cliente	apellido	Es el apellido del cliente	varchar	50
65	Dni del cliente	dni	Es el dni del cliente	int	8
66	Email del cliente	email	Es el email del cliente	varchar	50
67	Telefono del cliente	telefono	Es el telefono del cliente	int	10
68	Direccion del cliente	dire	Es la direccion del cliente	varchar	1000
69	Codigo postal	cod_postal	Es el codigo postal del cliente	int	4



	A	B	C	D	E
70	Empleados				
71	Nombre	Alias	Descripcion	Tipo	Longitud
72	Identificador del empleado	ID_empleado PK	Es el id del empleado	int unsigned auto_increment	11
73	Nombre del empleado	nombre	Es el nombre del empleado	varchar	50
74	Apellido del empleado	apellido	Es el apellido del empleado	varchar	50
75	Dni del empleado	dni	Es el dni del empleado	int	8
76	Email del empleado	email	Es el email del empleado	varchar	100
77	Telefono del empleado	telefono	Es el telefono del empleado	int	10
78	Sector del empleado	sector	Es el sector del empleado	varchar	100
79	Cargo del empleado	cargo	Es el cargo del empleado	varchar	100
80	Rol del empleado	rol	Es el rol del empleado	varchar	100
81					
82	Reportes				
83	Nombre	Alias	Descripcion	Tipo	Longitud
84	Identificador del reporte	ID_reporte PK	Es el id de reporte	int unsigned auto_increment	11
85	Detalle	detalle	Es el detalle del reporte	varchar	10000
86	Reportes_Ticket				
87	Nombre	Alias	Descripcion	Tipo	Longitud
88	Identificador del detalle	Id_rt PK	es el identificador del detalle	int unsigned auto_increment	11
89	Identificador del reporte	id_reporte FK	Es el identificador del reporte	int unsigned	11
90	Identificador del ticket	id_ticket FK	Es el identificador del ticket	int unsigned	11
91					
92	Ticket				
93	Nombre	Alias	Descripcion	Tipo	Longitud
94	Identificador del ticket	ID_ticket PK	es el identificador del ticket	int unsigned auto_increment	11
95	Identificador del cliente	ID_cliente FK	Es el id del cliente	int unsigned	11
96	Fecha de realizacion	fecha	Es la fecha en la que se hizo la transaccion	date	-
97	Detalle del ticket	detalle	Es el detalle del ticket	varchar	1000
98	Total del ticket	total	Es el total del tiket	int	11
99	MetodoPago	metodospago	Es el metodo de pago con el que se hizo	varchar	100
100					

	A	B	C	D	E
101	Almacenes				
102	Nombre	Alias	Descripcion	Tipo	Longitud
103	Identificador del almacen	id_almacen PK	Identificador del almacen	int unsigned auto_increment	11
104	Diraccion del almacen	direccion	Es la direccion del almacen	varchar	50
105	Ciudad del almacen	ciudad FK	Es la ciudad del almacen	int unsinged	11
106	Nombre del almacen	nombre	Es el nombre del almacen	varchar	50
107					
108	Ciudades				
109	Nombre	Alias	Descripcion	Tipo	Longitud
110	Identificador de la ciudad	ID_ciudad PK	Es el id de la ciudad	int unsigned auto_increment	11
111	Nombre de la ciudad	Nombre	Es el nombre de la ciudad	varchar	50
112	Identificador de la provincia	provincia	Es la provincia donde se ubica la ciudad	varchar	50
113					
114					
115					
116					
117					
118					
119					
120					
121					
122					
123					
124					
125					
126					
127					
128					
129					
130					
131					

4/9

me encargué de realizar distintas vistas en SQL que me fueron asignadas para el proyecto, junto con mi compañero Benjamín Saragusti.

Una vista SQL es una consulta guardada dentro de la base de datos que permite mostrar información de una o varias tablas. Su utilidad es facilitar el acceso a determinados datos sin tener que escribir la misma consulta cada vez que se necesita obtener esa información.

En nuestro proyecto decidimos crear distintas vistas para poder consultar información importante de la empresa logística de una manera más organizada.

Las vistas que desarrollé fueron las siguientes:

1. Vehículos con empleados

Esta vista permite consultar la información relacionada con los vehículos y los empleados asociados a ellos. De esta manera se puede tener una referencia de qué empleados están relacionados con los vehículos de la empresa.

2. Envíos no entregados

Esta vista permite consultar los envíos que todavía no fueron entregados. Puede resultar útil para que los administradores puedan identificar cuáles son los pedidos que todavía necesitan seguimiento.

3. Envíos cancelados

Esta vista permite consultar los envíos que fueron cancelados. Así se puede acceder a esa información sin tener que revisar todos los envíos registrados en la base de datos.

4. Empleados con rol administrador

Esta vista permite mostrar únicamente a los empleados que tienen asignado el rol de administrador. Esto facilita la consulta de las personas que cuentan con ese cargo dentro de la empresa.

5. Productos con stock bajo

Esta vista permite identificar los productos cuya cantidad disponible se encuentra por debajo del nivel esperado. Esto puede ayudar a la empresa a detectar cuándo necesita reponer determinados productos.

6. Paquetes pesados

Esta vista permite consultar los paquetes que cumplen con la condición de tener un peso elevado. Puede ser útil para identificarlos y tener en cuenta sus características al momento de organizar los envíos.

La realización de estas vistas me permitió trabajar con consultas SQL y comprender cómo se puede organizar la información de una base de datos para facilitar su consulta.

```
create view vehiculo_empleado as select vehiculos.ID_Vehiculo, vehiculos.Marca, vehiculos.Modelo, empleados.Nombre,empleados.Apellido,empleados.Dni from vehiculos left join empleados on empleados.ID_Empleado = vehiculos.ID_Empleado;

create view envios_noentregados as select * from envios
where estado = "NO ENTREGADO";

create view envios_cancelados as select * from envios
where estado = "CANCELADO";

create view administradores as select * from empleados
where empleados.rol = "ADMINISTRADOR";

create view Productos_critico as select * from productos
where productos.stock <10;

create view Paquetes_pesados as select * from paquetes
where paquetes.peso <100;
```

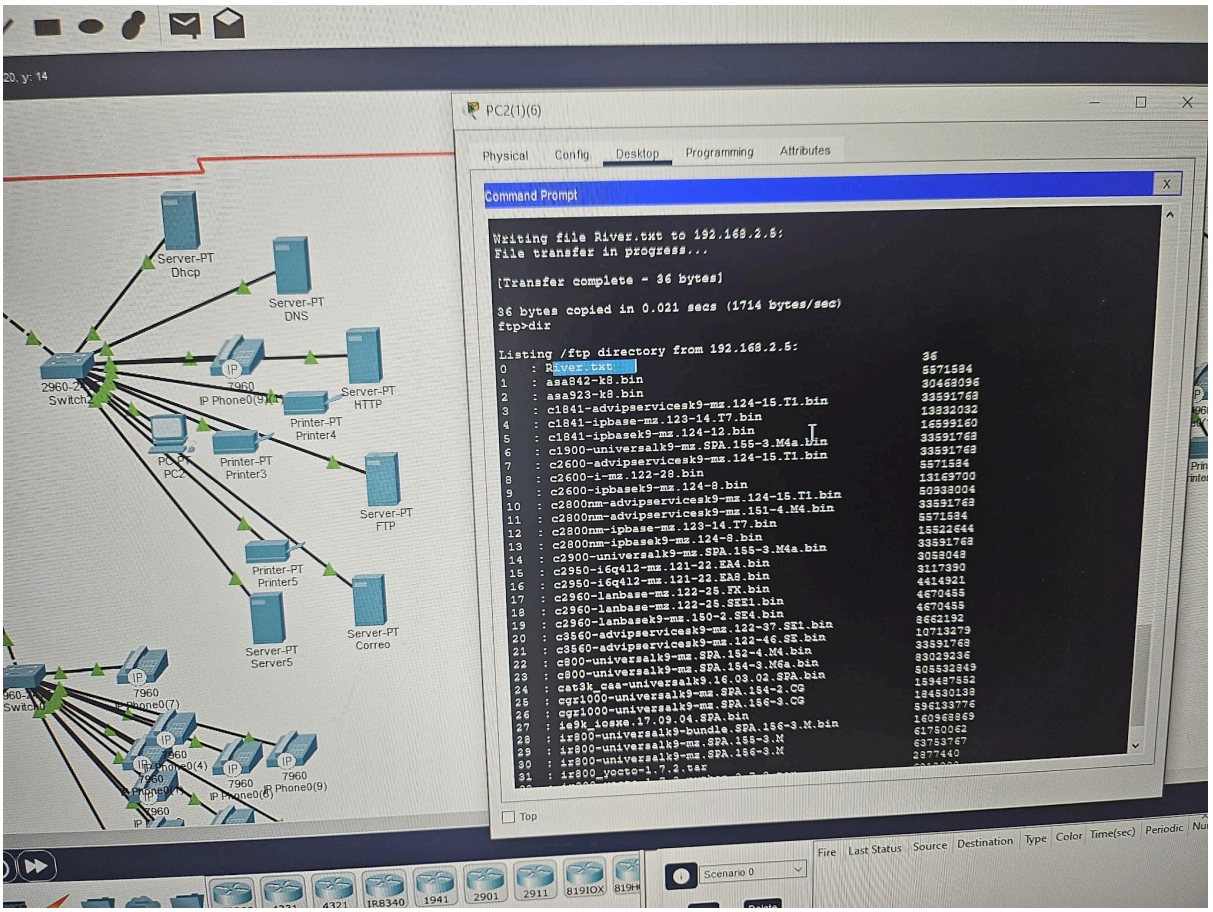
foto sacada del github, estas vistas las hice con la ayuda de benjamín saragusti

10/9

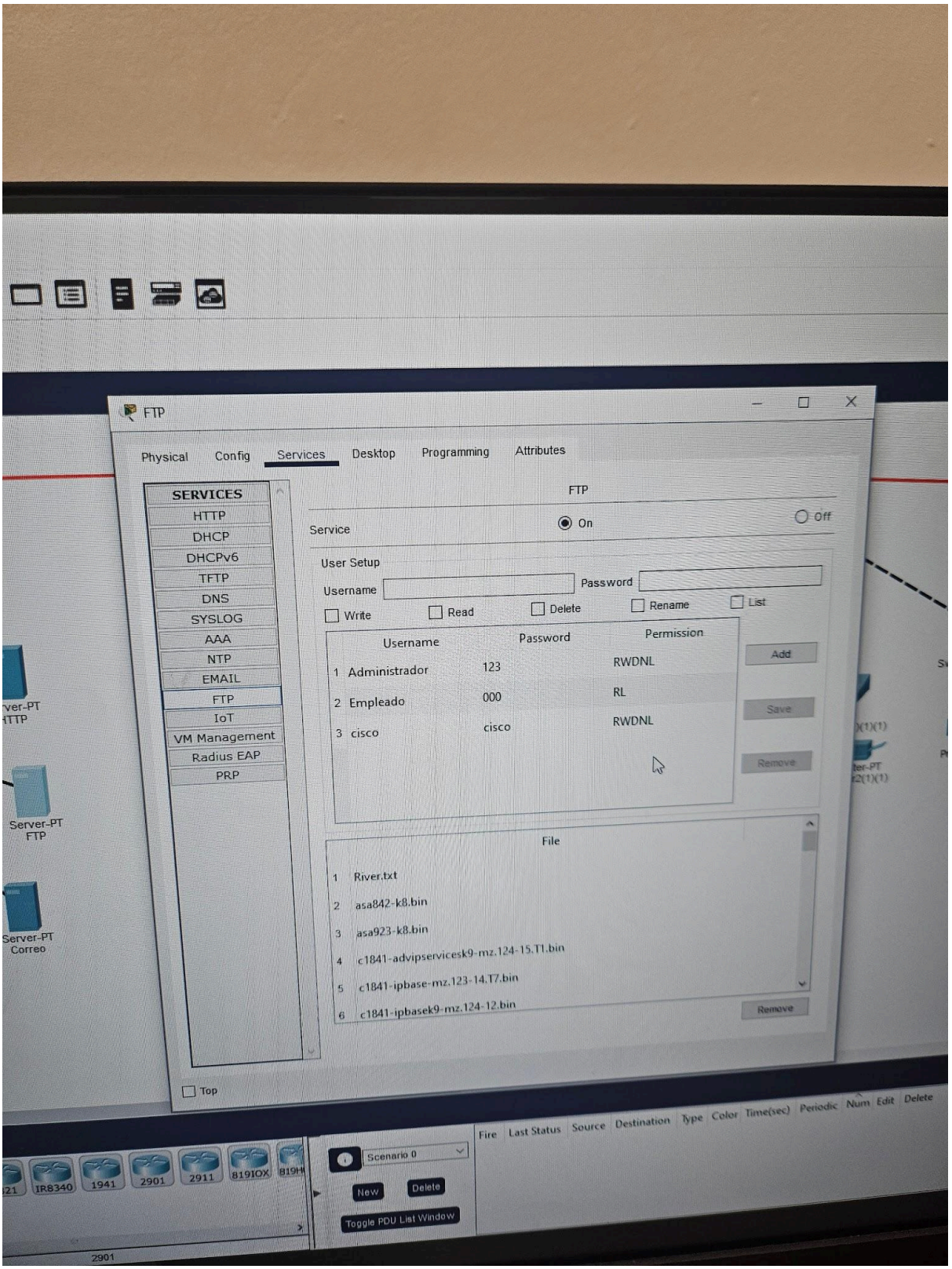
En esta fecha, luego de que el profesor nos explicara la teoría sobre los servidores FTP, me encargué, junto con mi compañero Saragusti, de realizar las pruebas de transferencia de archivos entre las distintas redes de nuestra empresa.

Como primer paso, configuramos dos tipos de usuarios. Por un lado, creamos un usuario administrador, al que le otorgamos todos los permisos disponibles. Por otro lado, creamos un usuario empleado, que solamente tenía permitido leer y listar los archivos. Para crear ambos usuarios y sus respectivas contraseñas, recurrimos a la IA para que nos proporcionara algunas opciones.

Una vez configurados los usuarios, pasamos a realizar la prueba de transferencia. Para esto, creamos un archivo de texto (.txt) llamado River en una de las máquinas. Luego, utilizamos el servidor FTP para enviarlo hacia un servidor ubicado en otra red, con el objetivo de comprobar que la transferencia de archivos funcionara correctamente.



Como se puede observar en la siguiente imagen, el archivo fue transferido correctamente al servidor FTP. Allí se puede ver que el archivo de texto llamado River se encuentra dentro del servidor, lo que nos permitió comprobar que el envío entre las distintas redes se había realizado correctamente.


FTP
Physical
Config
Services
Desktop
Programming
Attributes
SERVICES
HTTP
DHCP
DHCPv6
TFTP
DNS
SYSLOG
AAA
NTP
EMAIL
FTP
IoT
VM Management
Radius EAP
PRP
FTP
Service
On
Off
User Setup
Username
Password
Write
Read
Delete
Rename
List
Username
Password
Permission
1 Administrador 123 RWDNL
2 Empleado 000 RL
3 cisco cisco RWDNL
Add
Save
Remove
File
1 River.txt
2 asa842-k8.bin
3 asa923-k8.bin
4 c1841-advipservicesk9-mz.124-15.T1.bin
5 c1841-ipbase-mz.123-14.T7.bin
6 c1841-ipbasek9-mz.124-12.bin
Remove
Top
Server-PT FTP
Server-PT Correo
IR8340
1941
2901
2911
819IOX
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Source
Destination
Type
Color
Time(sec)
Periodic
Num
Edit
Delete

13/9

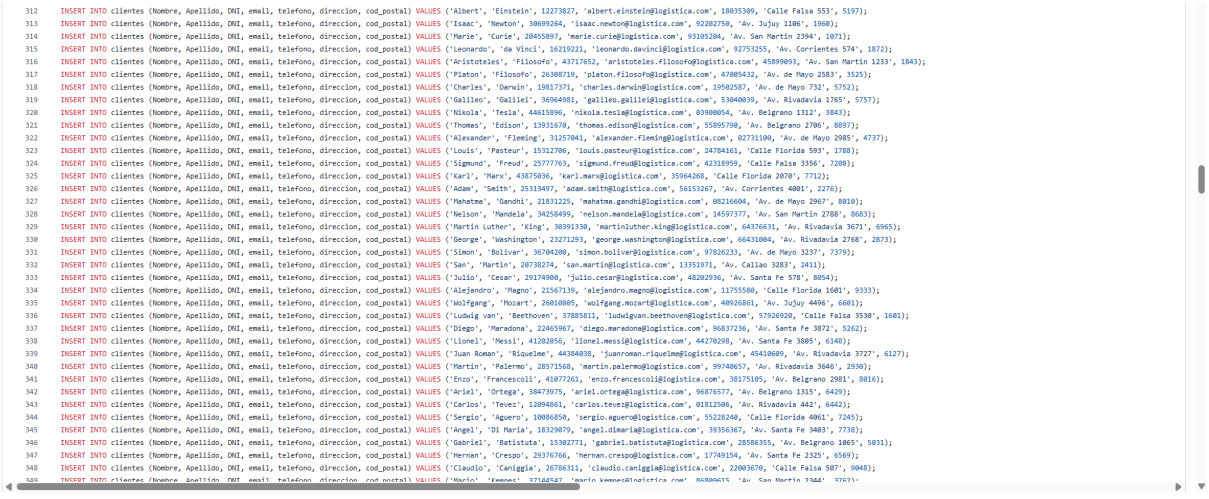
En esta fecha, me dispuse a realizar el trabajo que se me había asignado el día 9/9, que consistía en desarrollar los procedimientos almacenados de la tabla **Almacenes**.

Al principio tuve algunas dificultades porque no recordaba bien cómo funcionaban estos procedimientos. Por ese motivo, decidí volver a repasar la teoría para comprender mejor cómo se creaban y de qué manera podían utilizarse en nuestra base de datos. Una vez que tuve más claros los conceptos, comencé a desarrollar los procedimientos correspondientes a mi tabla.

21/9

En esta fecha, me dispuse a agregar registros a la tabla Clientes mediante sentencias INSERT. Esta tarea no presentó demasiadas dificultades, ya que consistía principalmente en completar los campos correspondientes de la tabla con los datos necesarios para registrar nuevos clientes en la base de datos.

De esta manera, pude incorporar información de prueba y contar con registros que luego podrían utilizarse para realizar consultas y comprobar el funcionamiento del sistema.



```
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Albert', 'Einstein', 12273827, 'albert.einstein@logistica.com', 18035309, 'Calle Falsa 553', 5197);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Isaac', 'Newton', 30699264, 'isaac.newton@logistica.com', 92202758, 'Av. Jujuy 1106', 1960);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Marie', 'Curie', 20455897, 'marie.curie@logistica.com', 93105204, 'Av. San Martín 2394', 1071);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Leonardo', 'da Vinci', 16219221, 'leonardo.davinci@logistica.com', 92753255, 'Av. Corrientes 574', 1872);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Aristoteles', 'Filosofo', 43717652, 'aristoteles.filosofo@logistica.com', 45899093, 'Av. San Martín 1233', 1843);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Platon', 'Filosofo', 26308719, 'platon.filosofo@logistica.com', 47005432, 'Av. de Mayo 2583', 3525);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Charles', 'Darwin', 19817371, 'charles.darwin@logistica.com', 19502587, 'Av. de Mayo 732', 5752);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Galileo', 'Galilei', 36964981, 'galileo.galilei@logistica.com', 53040039, 'Av. Rivadavia 1765', 5757);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Nikola', 'Tesla', 44615896, 'nikola.tesla@logistica.com', 03900054, 'Av. Belgrano 1312', 3843);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Thomas', 'Edison', 13931670, 'thomas.edison@logistica.com', 55895790, 'Av. Belgrano 2706', 8897);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Alexander', 'Fleming', 31257041, 'alexander.fleming@logistica.com', 02731100, 'Av. de Mayo 2985', 4737);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Louis', 'Pasteur', 15312706, 'louis.pasteur@logistica.com', 24784161, 'Calle Florida 593', 1788);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Sigmund', 'Freud', 25777763, 'sigmund.freud@logistica.com', 42318959, 'Calle Falsa 3356', 7208);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Karl', 'Marx', 43875036, 'karl.marx@logistica.com', 35964268, 'Calle Florida 2070', 7712);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Adam', 'Smith', 25313497, 'adam.smith@logistica.com', 56153267, 'Av. Corrientes 4001', 2276);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Mahatma', 'Gandhi', 21831225, 'mahatma.gandhi@logistica.com', 00216604, 'Av. de Mayo 2967', 8010);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Nelson', 'Mandela', 34258499, 'nelson.mandela@logistica.com', 14597377, 'Av. San Martín 2788', 8683);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Martin Luther', 'King', 30391330, 'martinluther.king@logistica.com', 64376631, 'Av. Rivadavia 3671', 6965);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('George', 'Washington', 23271293, 'george.washington@logistica.com', 66431004, 'Av. Rivadavia 2768', 2873);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Simon', 'Bolivar', 36704200, 'simon.bolivar@logistica.com', 97826233, 'Av. de Mayo 3237', 7379);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('San', 'Martin', 20738274, 'san.martin@logistica.com', 13351971, 'Av. Callao 3283', 2411);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Julio', 'Cesar', 29174900, 'julio.cesar@logistica.com', 48202936, 'Av. Santa Fe 578', 8054);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Alejandro', 'Magno', 21567139, 'alejandro.magno@logistica.com', 11755580, 'Calle Florida 1601', 9333);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Wolfgang', 'Mozart', 26010805, 'wolfgang.mozart@logistica.com', 40926861, 'Av. Jujuy 4496', 6601);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ludwig van', 'Beethoven', 37885811, 'ludwigvan.beethoven@logistica.com', 57926920, 'Calle Falsa 3530', 1601);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Diego', 'Maradona', 22465967, 'diego.maradona@logistica.com', 96837236, 'Av. Santa Fe 3872', 5262);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Lionel', 'Messi', 41202056, 'lionel.messi@logistica.com', 44270298, 'Av. Santa Fe 3805', 6140);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Juan Roman', 'Riquelme', 44384038, 'juanroman.riquelme@logistica.com', 45410609, 'Av. Rivadavia 3727', 6127);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Martin', 'Palermo', 28571568, 'martin.palermo@logistica.com', 99740657, 'Av. Rivadavia 3646', 2930);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Enzo', 'Francescoli', 41077261, 'enzo.francescoli@logistica.com', 38175105, 'Av. Belgrano 2981', 8016);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ariel', 'Ortega', 38473975, 'ariel.ortega@logistica.com', 96876577, 'Av. Belgrano 1315', 6429);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Carlos', 'Tevez', 12094861, 'carlos.tevez@logistica.com', 01812506, 'Av. Rivadavia 442', 6442);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Sergio', 'Aguero', 10086850, 'sergio.aguero@logistica.com', 55228240, 'Calle Florida 4061', 7245);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Angel', 'Di Maria', 18329079, 'angel.dimaria@logistica.com', 39356367, 'Av. Santa Fe 3403', 7738);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Gabriel', 'Batistuta', 15302771, 'gabriel.batistuta@logistica.com', 28586355, 'Av. Belgrano 1865', 5031);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Hernan', 'Crespo', 29376766, 'hernan.crespo@logistica.com', 17749154, 'Av. Santa Fe 2325', 6569);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Claudio', 'Caniggia', 26786311, 'claudio.caniggia@logistica.com', 22003670, 'Calle Falsa 507', 9048);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Mario', 'Kempes', 37144547, 'mario.kempes@logistica.com', 86809615, 'Av. San Martín 2344', 3762);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Daniel', 'Passarella', 10805063, 'daniel.passarella@logistica.com', 28775364, 'Av. Rivadavia 3666', 6539);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ubaldo', 'Fillol', 38922226, 'ubaldo.fillol@logistica.com', 21576356, 'Calle Florida 4375', 6217);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Ricardo', 'Bochini', 43561837, 'ricardo.bochini@logistica.com', 33084111, 'Av. Santa Fe 2310', 9079);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Norberto', 'Alonso', 38242606, 'norberto.alonso@logistica.com', 99419529, 'Av. Callao 362', 2323);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Rene', 'Houseman', 18600586, 'rene.houseman@logistica.com', 74545965, 'Av. de Mayo 4391', 6759);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Javier', 'Zanetti', 42229798, 'javier.zanetti@logistica.com', 63119857, 'Av. Santa Fe 1995', 1878);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Diego', 'Simeone', 34412200, 'diego.simeone@logistica.com', 83718168, 'Av. de Mayo 2009', 8944);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Juan Sebastian', 'Veron', 20657150, 'juansebastian.veron@logistica.com', 13882685, 'Av. de Mayo 2364', 6520);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Fernando', 'Redondo', 13131666, 'fernando.redondo@logistica.com', 12206316, 'Av. San Martín 2612', 2446);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Javier', 'Mascherano', 29971903, 'javier.mascherano@logistica.com', 92421769, 'Calle Florida 3740', 2141);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Gonzalo', 'Higuain', 15363821, 'gonzalo.higuain@logistica.com', 06635937, 'Av. Santa Fe 981', 1101);
INSERT INTO clientes (Nombre, Apellido, DNI, email, telefono, direccion, cod_postal) VALUES ('Julian', 'Alvarez', 33896037, 'julian.alvarez@logistica.com', 04766667, 'Av. Corrientes 3923', 2859);
```

fotos sacadas del git hub

6/10

En esta etapa del proyecto, terminé de desarrollar los procedimientos almacenados correspondientes a la tabla Almacenes. Para realizarlos, primero tuve que pensar qué operaciones podían resultar útiles para administrar la información de esta tabla y facilitar su manejo desde la base de datos.

A partir de eso, se me ocurrió implementar distintos procedimientos que permitieran agregar, modificar, eliminar, mostrar y buscar almacenes, además de ordenarlos alfabéticamente de forma ascendente o descendente. De esta manera, pude completar las operaciones principales necesarias para gestionar los registros de la tabla.

1 procedimiento que ordene los nombres de los almacenes de la A a la Z

Este procedimiento permite mostrar los almacenes ordenados alfabéticamente según su nombre. Para realizarlo utilicé el comando ASC, que permite ordenar los resultados de forma ascendente.

1 procedimiento que ordene los nombres de los almacenes de la Z a la A

Este procedimiento cumple una función similar al anterior, pero muestra los almacenes en orden alfabético inverso. Para esto utilicé el comando DESC, que permite ordenar los resultados de forma descendente.

procedimiento agregar almacen

Permite modificar la información de un almacén que ya se encuentra registrado.

procedimiento editar almacen

procedimiento eliminar almacen

Permite eliminar un almacén de la base de datos.

procedimiento mostrar almacenes

Permite consultar todos los registros que se encuentran guardados en la tabla almacenes.

procedimiento buscar almacen por ID

Permite encontrar un almacén específico utilizando su identificador (ID), lo que facilita la búsqueda de un registro determinado.

Para realizar los procedimientos utilicé diferentes instrucciones de SQL, como SELECT, ORDER BY, ASC y DESC, dependiendo de la operación que necesitaba realizar.

Este trabajo me permitió comprender mejor cómo funcionan los procedimientos almacenados y cómo pueden utilizarse para organizar y simplificar las operaciones dentro de una base de datos.

```
DELIMITER //
CREATE PROCEDURE almacenes_AZ ()
BEGIN
select * from almacenes
order by nombre ASC;
end;
delimiter ;
```

```
DELIMITER //
CREATE PROCEDURE almacenes_ZA ()
BEGIN
select * from almacenes
order by nombre DESC;
end;
delimiter ;
```

AGREGAR ALMACEN:

```
DELIMITER //

CREATE PROCEDURE AgregarAlmacen(
   IN p_direccion VARCHAR(50),
   IN p_ciudad INT,
   IN p_nombre VARCHAR(50)
)
BEGIN
   IF EXISTS (SELECT 1 FROM Ciudades WHERE ID_ciudad = p_ciudad) THEN

      INSERT INTO Almacenes (direccion, ciudad, nombre)
      VALUES (p_direccion, p_ciudad, p_nombre);

   ELSE
```

EDITAR UN ALMACEN:

*

```
DELIMITER //

CREATE PROCEDURE EditarAlmacen(
   IN p_id_almacen INT,
   IN p_direccion VARCHAR(50),
   IN p_ciudad INT,
   IN p_nombre VARCHAR(50)
)
BEGIN
   UPDATE Almacenes
   SET direccion = p_direccion,
      ciudad = p_ciudad,
      nombre = p_nombre
   WHERE id_almacen = p_id_almacen;
END //

DELIMITER ;
```

BORRAR ALMACÉN:

```
DELIMITER //

CREATE PROCEDURE BorrarAlmacen(
   IN p_id_almacen INT
)
BEGIN
   DELETE FROM Almacenes
   WHERE id_almacen = p_id_almacen;
END //
```

MOSTRAR TODOS LOS ALMACENES

```
DELIMITER //
create procedure mostrar_almacen()
BEGIN
        select *
   from almacenes;
end;
DELIMITER ;
```

BUSCAR UN ALMACÉN

```
DELIMITER //
create PROCEDURE buscar_almacen(in id int)
BEGIN
        SELECT
   from almacenes
   where id = almacenes.ID_Almacen
   end;
        DELIMITER ;
```

16/10

El día de hoy me dispuse a realizar los triggers correspondientes a mi tabla almacenes. En total desarrollé cinco triggers, con el objetivo de controlar los datos que se ingresan o modifican y evitar que se guarde información incorrecta en la base de datos.

Los triggers son instrucciones que se ejecutan automáticamente cuando ocurre una determinada acción en una tabla. En este caso, los utilicé para establecer condiciones que deben cumplirse antes de permitir determinadas operaciones.

Los cinco triggers que realicé fueron los siguientes:

1)EVITAR NOMBRES DE ALMACEN VACIOS AL INSERTAR

Este trigger controla que, al intentar registrar un nuevo almacén, el campo correspondiente al nombre no se encuentre vacío. De esta manera se evita guardar un almacén sin identificación.

2)EVITAR DIRECCIONES VACIAS

Este trigger comprueba que el campo de dirección tenga información al momento de insertar un almacén. Esto permite evitar registros que no tengan una ubicación especificada.

3)EVITAR NOMBRES VACIOS AL MODIFICAR

Este trigger cumple una función similar al primero, pero se aplica cuando se intenta modificar un almacén que ya existe. Su objetivo es impedir que se actualice un registro dejando su nombre vacío.

4)EVITAR DIRECCIONES VACIAS AL MODIFICAR

Este trigger controla que, al modificar los datos de un almacén, no se deje el campo de dirección sin completar.

5)Evitar que el almacen se mueva a una ciudad inexistente

Este trigger verifica que la ciudad asignada al almacén exista dentro de la base de datos. Esto permite evitar que un almacén quede relacionado con una ciudad que no se encuentra registrada.

La realización de estos triggers me permitió comprender cómo se pueden utilizar las condiciones automáticas dentro de una base de datos para controlar la información. También aprendí que es importante validar los datos antes de guardarlos, ya que un registro incorrecto puede generar problemas en otras partes del sistema.

los triggers quedaron tal que así:
1)

```
DELIMITER //

CREATE TRIGGER validar_nombre_almacen
BEFORE INSERT ON Almacenes
FOR EACH ROW
BEGIN
    IF NEW.nombre IS NULL OR TRIM(NEW.nombre) = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El nombre del almacen no puede estar vacio';
    END IF;
END //
DELIMITER;

DELIMITER //
```

2)

```
13
14      DELIMITER //
15
16      CREATE TRIGGER validar_direccion_almacen
17      BEFORE INSERT ON Almacenes
18      FOR EACH ROW
19      BEGIN
20          IF NEW.direccion IS NULL OR TRIM(NEW.direccion) = '' THEN
21              SIGNAL SQLSTATE '45000'
22              SET MESSAGE_TEXT = 'La direccion no puede estar vacia';
23          END IF;
24      END //
25
26      DELIMITER ;
```

3)

```
27
28    DELIMITER //
29
30    CREATE TRIGGER validar_nombre_almacen_update
31    BEFORE UPDATE ON Almacenes
32    FOR EACH ROW
33    BEGIN
34        IF NEW.nombre IS NULL OR TRIM(NEW.nombre) = '' THEN
35            SIGNAL SQLSTATE '45000'
36            SET MESSAGE_TEXT = 'El nombre del almacen no puede estar vacio';
37        END IF;
38    END //
39
40    DELIMITER ;
```

4)

```
41
42      DELIMITER //
43
44      CREATE TRIGGER validar_direccion_almacen_update
45      BEFORE UPDATE ON Almacenes
46      FOR EACH ROW
47      BEGIN
48          IF NEW.direccion IS NULL OR TRIM(NEW.direccion) = '' THEN
49              SIGNAL SQLSTATE '45000'
50              SET MESSAGE_TEXT = 'La direccion no puede estar vacia';
51          END IF;
52      END //
53
54      DELIMITER ;
55
```

5)

```
55
56      DELIMITER //
57
58      CREATE TRIGGER validar_ciudad_almacen
59      BEFORE INSERT ON Almacenes
60      FOR EACH ROW
61      BEGIN
62          IF NOT EXISTS (
63              SELECT 1
64              FROM Ciudades
65              WHERE ID_Ciudad = NEW.ciudad
66          ) THEN
67              SIGNAL SQLSTATE '45000'
68              SET MESSAGE_TEXT = 'La ciudad indicada no existe';
69          END IF;
70      END //
71
72      DELIMITER ;
```

21/10

Una de las partes en las que trabajé durante el proyecto fue el desarrollo de las páginas relacionadas con los clientes. El objetivo era que tanto el administrador como el usuario pudieran realizar distintas operaciones relacionadas con los clientes registrados en nuestra empresa.

Primero me encargué de realizar la página para registrar clientes. Para hacerla tomé como referencia la página que ya teníamos para registrar paquetes, ya que ambas tenían una estructura parecida. Lo que hice fue cambiar los campos del formulario para que coincidieran con los datos que necesitábamos guardar de un cliente, como el nombre, apellido, DNI, dirección, código postal, correo electrónico y contraseña.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title> Registrar Cliente </title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/css/boo
tstrap.min.css" rel="stylesheet"
integrity="sha384-B/GM4XqrwHnWXNOWMbloTmrYXZg10cakYGmpfsR/bbzQ6JAJI4ihu
yADKLnBgrCe" crossorigin="anonymous">

   <script type="module"
src="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/js/boots
trap.bundle.min.js"
integrity="sha384-1a/pXj49ZQ1aHEmrJ+gMw1otqoVsYwlEnlD8mIfY2TV03r20Y0CN7
uqx1tQogjPL" crossorigin="anonymous"></script>
    <link rel="stylesheet" href="general.css">
</head>
<body class="fondo-blanco">

    <div class="fondo-blanco">

     <!-- BARRA DE BUSQUEDA -->
    <nav class="navbar fondo-blanco">
        <div class="container-fluid">
            <a class="navbar-brand texto-negro"
onclick="window.open('index.html')">

            Logistica
            </a>
            <li class="nav-item">
                <button type="button" class="btn hover-lift"
aria-current="page" href="#" onclick="window.open('VerEnvios.html',
'_self');">
                  <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-box" viewBox="0 0 16 16">
                  <path d="M8.186 1.113a.5.5 0 0 0-.372 0L1.846 3.5 8
5.961 14.154 3.5zM15 4.239l-6.5 2.6v7.922l6.5-2.6V4.24zM7.5
14.762V6.838L1 4.239v7.923zM7.443.184a1.5 1.5 0 0 1 1.114 0l7.129
```

```
2.852A.5.5 0 0 1 16 3.5v8.662a1 1 0 0 1-.629.928l-7.185 2.874a.5.5 0 0
1-.372 0L.63 13.09a1 1 0 0 1-.63-.928V3.5a.5.5 0 0 1 .314-.464z"/>
                </svg>
                Ver Clientes</a>
            </li>
            <li class="nav-item">
                <button type="button" class="btn hover-lift"
onclick="alert('Numero de  telefono : +54 911 6767 6767 \nCorreo
electronico : logistica32@gmail.com  \nNombre de contacto:Logistica 32
Equipo 6')">
                <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-telephone" viewBox="0 0 16
16">
                <path d="M3.654 1.328a.678.678 0 0 0-1.015-.063L1.605
2.3c-.483.484-.661 1.169-.45 1.77a17.6 17.6 0 0 0 4.168 6.608 17.6 17.6
0 0 0 6.608 4.168c.601.211 1.286.033 1.77-.45l1.034-1.034a.678.678 0 0
0-.063-1.015l-2.307-1.794a.68.68 0 0 0-.58-.122l-2.19.547a1.75 1.75 0 0
1-1.657-.459L5.482 8.062a1.75 1.75 0 0 1-.46-1.657l.548-2.19a.68.68 0 0
0-.122-.58zM1.884.511a1.745 1.745 0 0 1 2.612.163L6.29
2.98c.329.423.445.974.315 1.494l-.547 2.19a.68.68 0 0 0 .178.643l2.457
2.457a.68.68 0 0 0 .644.178l2.189-.547a1.75 1.75 0 0 1 1.494.315l2.306
1.794c.829.645.905 1.87.163 2.611l-1.034 1.034c-.74.74-1.846
1.065-2.877.702a18.6 18.6 0 0 1-7.01-4.42 18.6 18.6 0 0
1-4.42-7.009c-.362-1.03-.037-2.137.703-2.877z"/>
                </svg>
                Contacto</button>
            </li>
            <button id="nombre_inicio" type="button" class="btn
hover-lift" aria-current="page"
onclick="window.open('Inicio_sesion.html', '_self');">
              <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-person" viewBox="0 0 16
16">
              <path d="M8 8a3 3 0 1 0 0-6 3 3 0 0 0 0 6m2-3a2 2 0 1 1-4
0 2 2 0 0 1 4 0m4 8c0 1-1 1-1 1H3s-1 0-1-1 1-4 6-4 6 3 6
4m-1-.004c-.001-.246-.154-.986-.832-1.664C11.516 10.68 10.289 10 8
10s-3.516.68-4.168 1.332c-.678.678-.83 1.418-.832 1.664z"/>
              </svg>
              Iniciar sesion</button>
        </div>
    </nav>

    <div class="separador">
```

```
    </div>

    <h1 class="texto-negro"> formulario para registrar clientes</h1>

    <br>

      <div class="container mt-5 ">
      <div class="row justify-content-center">
        <div class="col-md-5 col-lg-3 test_form">
          <div class="card shadow">
            <div class="card-header ">
              <h4 class="texto-formulario mb-0 text-center
texto-blanco" >Formulario de Registro</h4>
            </div>
            <div class="card-body">
              <form>
                <div class="mb-3">
                  <label for="nombre" class="form-label">Nombre</label>
                  <input type="text" class="form-control" id="nombre"
required>
                </div>


                <div class="mb-3">
                  <label for="nombre"
class="form-label">Apellido</label>
                  <input type="text" class="form-control" id="apellido"
required>
                </div>

                <div class="mb-3">
                  <label for="email" class="form-label">Email</label>
                  <input type="text" class="form-control" id="email"
required>
                </div>

                <div class="mb-3">
                  <label for="contrasenia"
class="form-label">Contrasenia</label>
                  <input type="text" class="form-control"
id="contrasenia" required>
                </div>
```

```
                <button type="submit" class="btn btn-primary w-100
hover-lift">Enviar</button>
              </form>
            </div>
          </div>
        </div>
      </div>



    </div>
```

cambié el nombre de los datos que requieren para registrar un paquete a los que se requieren para los clientes. luego cambié el icono que tenia la sección “ver clientes” de una caja que tenia a una especie de icono dentro de un marco, me guié usando bootstrap para cambiar el icono, quedando así:

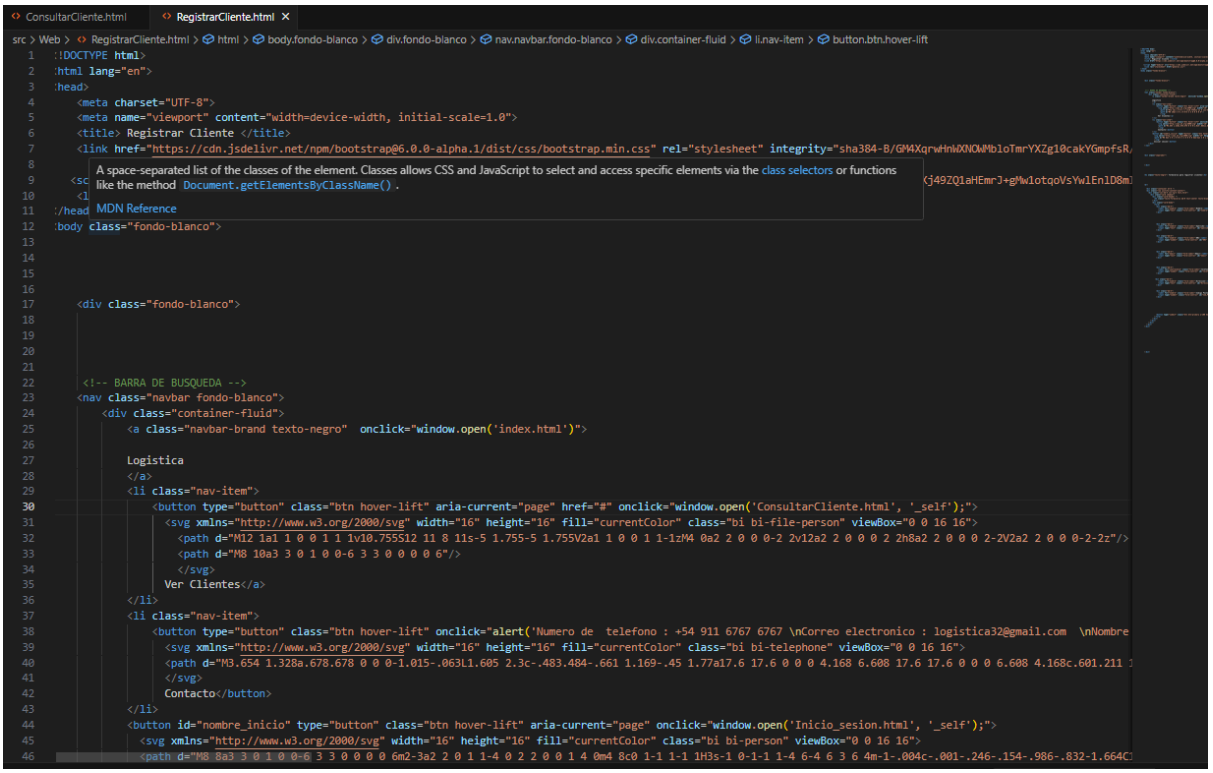
```
 <li class="nav-item">
                <button type="button" class="btn hover-lift"
aria-current="page" href="#" onclick="window.open('VerEnvios.html',
'_self');">
                  <svg xmlns="http://www.w3.org/2000/svg" width="16"
height="16" fill="currentColor" class="bi bi-file-person" viewBox="0 0
16 16">
                    <path d="M12 1a1 1 0 0 1 1 1v10.755S12 11 8 11s-5
1.755-5 1.755V2a1 1 0 0 1 1-1zM4 0a2 2 0 0 0-2 2v12a2 2 0 0 0 2 2h8a2 2
0 0 0 2-2V2a2 2 0 0 0-2-2z"/>
                    <path d="M8 10a3 3 0 1 0 0-6 3 3 0 0 0 0 6"/>
                    </svg>
                  Ver Clientes</a>
            </li>
```

Al revisar la página RegistrarCliente, me di cuenta de que el formulario estaba incompleto, ya que en la tabla de clientes había algunos campos que no se podían completar desde el código anterior. Un ejemplo de esto era el código postal.

Por este motivo, decidí realizar algunas modificaciones y agregar tres campos más al formulario: DNI, Dirección y Código Postal, para que la información ingresada coincidiera con los datos requeridos por la tabla de clientes.

Además, aproveché para corregir el funcionamiento del botón Ver clientes, modificando su atributo `onclick` para que, al presionarlo, redirigiera a la página donde se muestran los clientes que ya fueron registrados.

De esta manera, busqué mejorar el formulario y asegurarme de que permitiera ingresar los datos necesarios para registrar correctamente a cada cliente.



```
                </svg>
                Iniciar sesion</button>
            </div>
        </nav>




        <div class="separador">



        </div>



        <h1 class="texto-negro"> formulario para registrar clientes</h1>



        <br>
          <div class="container mt-5 ">
          <div class="row justify-content-center">
            <div class="col-md-5 col-lg-3 test_form">
              <div class="card shadow">
                <div class="card-header ">
                  <h4 class="texto-formulario mb-0 text-center texto-blanco" >Formulario de Registro</h4>
                </div>
                <div class="card-body">
                  <form>
                    <div class="mb-3">
                      <label for="nombre" class="form-label">Nombre</label>
                      <input type="text" class="form-control" id="nombre"  required>
                    </div>
```

```
                    <div class="mb-3">
                      <label for="nombre" class="form-label">Apellido</label>
                      <input type="text" class="form-control" id="apellido"  required>
                    </div>


                    <div class="mb-3">
                      <label for="nombre" class="form-label">DNI</label>
                      <input type="number" class="form-control" id="DNI"  required>
                    </div>

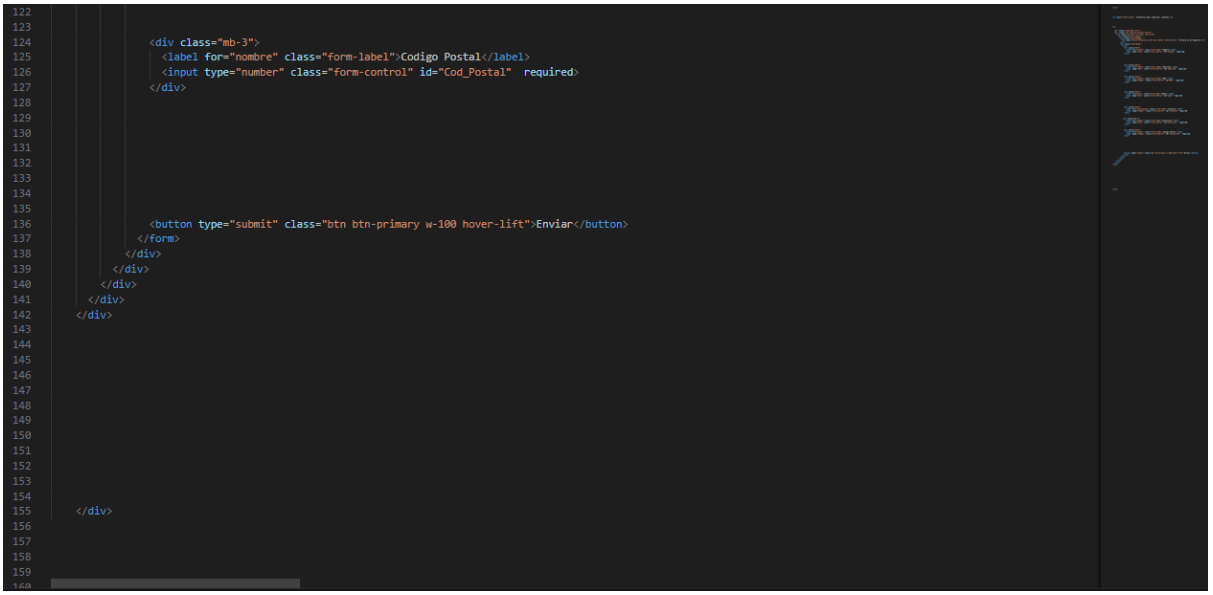


                    <div class="mb-3">
                      <label for="email" class="form-label">Email</label>
                      <input type="text" class="form-control" id="email" required>
                    </div>



                    <div class="mb-3">
                      <label for="contrasenia" class="form-label">telefono</label>
                      <input type="number" class="form-control" id="telefono" required>
                    </div>


                   <div class="mb-3">
                      <label for="nombre" class="form-label">Direccion</label>
                      <input type="text" class="form-control" id="Direccion"  required>
                    </div>
```



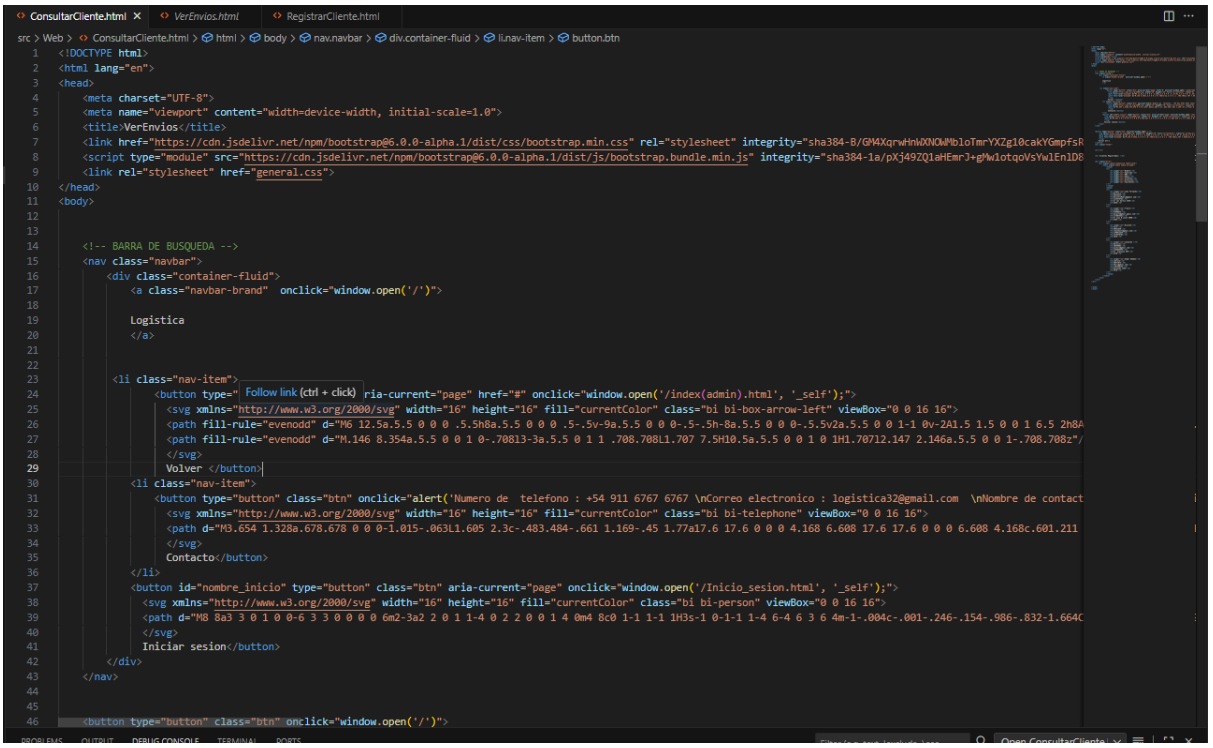
```
            <div class="mb-3">
              <label for="nombre" class="form-label">Codigo Postal</label>
              <input type="number" class="form-control" id="Cod_Postal"  required>
            </div>

            <button type="submit" class="btn btn-primary w-100 hover-lift">Enviar</button>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>

</div>
```

Luego, me encargué de desarrollar la página **ConsultaCliente**, cuya función es mostrar los clientes que ya se encuentran registrados en el sistema. Para realizarla, al igual que había hecho anteriormente con la página **RegistrarCliente**, tomé como referencia una página que ya estaba desarrollada. En este caso, utilicé **VerEnvios** como modelo y adapté su estructura para que pudiera mostrar la información correspondiente a los clientes.

De esta manera, pude reutilizar parte del trabajo que ya estaba hecho y adaptarlo a las necesidades de nuestro proyecto. Una vez realizados los cambios, el código quedó de la siguiente manera:



```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>VerEnvios</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-B/GM4XqrwHnWXNOWMbloTmrYXZg10cakYGmpfsR
    <script type="module" src="https://cdn.jsdelivr.net/npm/bootstrap@6.0.0-alpha.1/dist/js/bootstrap.bundle.min.js" integrity="sha384-1a/pXj49ZQ1aHEmrJ+gMwlotqoVsYwlEn1D8
    <link rel="stylesheet" href="general.css">
</head>
<body>


    <!-- BARRA DE BUSQUEDA -->
    <nav class="navbar">
        <div class="container-fluid">
            <a class="navbar-brand"  onclick="window.open('/')">

            Logistica
            </a>


        <li class="nav-item">
              <button type="
                Follow link (ctrl + click)
                ria-current="page" href="#" onclick="window.open('/index(admin).html', '_self');">
                <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-box-arrow-left" viewBox="0 0 16 16">
                <path fill-rule="evenodd" d="M6 12.5a.5.5 0 0 0 .5.5h8a.5.5 0 0 0 .5-.5v-9a.5.5 0 0 0-.5-.5h-8a.5.5 0 0 0-.5.5v2a.5.5 0 0 1-1 0v-2A1.5 1.5 0 0 1 6.5 2h8A
                <path fill-rule="evenodd" d="M.146 8.354a.5.5 0 0 1 0-.70813-3a.5.5 0 1 1 .708.708L1.707 7.5H10.5a.5.5 0 0 1 0 1H1.70712.147 2.146a.5.5 0 0 1-.708.708z"/
                </svg>
                Volver </button>
            <li class="nav-item">
                <button type="button" class="btn" onclick="alert('Numero de  telefono : +54 911 6767 6767 \nCorreo electronico : logistica32@gmail.com  \nNombre de contact
                <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-telephone" viewBox="0 0 16 16">
                <path d="M3.654 1.328a.678.678 0 0 0-1.015-.063L1.605 2.3c-.483.484-.661 1.169-.45 1.77a17.6 17.6 0 0 0 4.168 6.608 17.6 17.6 0 0 0 6.608 4.168c.601.211
                </svg>
                Contacto</button>
            </li>
            <button id="nombre_inicio" type="button" class="btn" aria-current="page" onclick="window.open('/Inicio_sesion.html', '_self');">
              <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-person" viewBox="0 0 16 16">
              <path d="M8 8a3 3 0 1 0 0-6 3 3 0 0 0 0 6m2-3a2 2 0 1 1-4 0 2 2 0 0 1 4 0m4 8c0 1-1 1-1 1H3s-1 0-1-1 1-4 6-4 6 3 6 4m-1-.004c-.001-.246-.154-.986-.832-1.664C
              </svg>
              Iniciar sesion</button>
        </div>
    </nav>


    <button type="button" class="btn" onclick="window.open('/')">
```

```
    <button type="button" class="btn" onclick="window.open('/')">
        <svg xmlns="http://www.w3.org/2000/svg" width="16" height="16" fill="currentColor" class="bi bi-box-arrow-left" viewBox="0 0 16 16">
        <path fill-rule="evenodd" d="M6 12.5a.5.5 0 0 0 .5.5h8a.5.5 0 0 0 .5-.5v-9a.5.5 0 0 0-.5-.5h-8a.5.5 0 0 0-.5.5v2a.5.5 0 0 1-1 0v-2A1.5 1.5 0 0 1 6.5 2h8A1.5 1.5 0
        <path fill-rule="evenodd" d="M.146 8.354a.5.5 0 0 1 0-.70813-3a.5.5 0 1 1 .708.708L1.707 7.5H10.5a.5.5 0 0 1 0 1H1.70712.147 2.146a.5.5 0 0 1-.708.708z"/>
        </svg>
        Volver atrás
    </button>
    <div class="fondo">


    <hr></hr>


    <h1> Clientes Registrados: </h1>


    <div class="mt-5">
        <div class="table-responsive table-card">
            <table class="table table-striped">
                <thead>
                <tr>
                    <th scope="col">Nombre</th>
                    <th scope="col">Apellido</th>
                    <th scope="col">DNI</th>
                    <th scope="col">Email</th>
                    <th scope="col">Telefono</th>
                    <th scope="col">Direccion</th>
                    <th scope="col">Cod_Postal</th>
                </tr>
                </thead>
                <tbody>
                <tr>
                    <th scope="row">Juan Fernando</th>
                    <td>Quintero</td>
                    <td>38123321</td>
                    <td>QuinteroMadrid@gmail.com</td>
                    <td>1124576761</td>
                    <td>av san martin 2454</td>
```

```
                    <td>1416</td>
                </tr>
                <tr>
                    <th scope="row">franco</th>
                    <td>armani</td>
                    <td>37890654</td>
                    <td>pulpito@gmail.gmail.com</td>
                    <td>1178675431</td>
                    <td>Av Juan B justo 3000</td>
                    <td>1445</td>
                </tr>
                <tr>
                    <th scope="row">Ricardo</th>
                    <td>fort</td>
                    <td>40561849</td>
                    <td>comandante@gmail.com</td>
                    <td>1190612335</td>
                    <td>1124576761</td>
                    <td>1416</td>
                </tr>
                <tr>
                    <th scope="row">Leonardo </th>
                    <td>Davinchi</td>
                    <td>10456082</td>
                    <td>pintura@gmail.com</td>
                    <td>1188664554</td>
                    <td>dr. espinosa 554</td>
                    <td>1416</td>
                </tr>
                <tr>
                    <th scope="row">Angel Amadeo</th>
                    <td>Labruna</td>
                    <td>20671073</td>
                    <td>Idolo@gmail.com</td>
                    <td>1161458123</td>
                    <td>paysandu 1245</td>
                    <td>1416</td>
                </tr>
```

```
                </tr>
                </tbody>
            </table>
        </div>
    </div>
</div>


</body>
</html>
```

Cambié los datos de la tabla envío por los de la tabla Cliente, y cambié el botón de ver envíos por el de volver. llevando de vuelta a la página main

Una vez terminadas las páginas, tuve que trabajar en las API para conectarlas con la base de datos. Para realizar esta parte me guié con las API que ya estaban hechas para paquetes, con la ayuda de Santiago Goldfarb y Benjamín Saragusti.

Las operaciones que desarrollé fueron las siguientes:

Registrar clientes: permite incorporar nuevos clientes a la base de datos.

Mostrar clientes: permite consultar los clientes que ya se encuentran registrados.

Editar clientes: permite modificar los datos de un cliente existente.

Eliminar clientes: permite quitar un cliente de los registros.

Durante este trabajo también tuve que adaptar partes del código que anteriormente pertenecían a la sección de paquetes, reemplazando los datos y las referencias para que funcionaran con la tabla de clientes.

Esta actividad me permitió trabajar con distintas partes del proyecto al mismo tiempo, ya que no solamente tuve que modificar las páginas HTML, sino también adaptar las API para que las operaciones pudieran realizarse con los datos correspondientes.



```
// -------------------------------- Clientes ---------------------------------------
        });
         servidor.createContext("/api/Clientes", new HttpHandler() {
            @Override
            public void handle(HttpExchange intercambio) throws IOException {
    String ruta = intercambio.getRequestURI().getPath();
    System.out.println("Contexto de Clientes: " + ruta);

    if (!ruta.equals("/api/Clientes")) {
        enviarRespuesta(intercambio, 404, "{\"error\":\"Ruta no encontrada\"}");
        return;
    }

    String sql = "call mostrar_clientes()";
    StringBuilder json = new StringBuilder();

    try (
        Connection conexion = DriverManager.getConnection("jdbc:mysql://localhost:3306/logistica32","root","");
        CallableStatement sentencia = conexion.prepareCall(sql);
        ResultSet resultado = sentencia.executeQuery()
    ) {
        json.append("[");
        boolean primero = true;

        while (resultado.next()) {
            if (!primero) {
                json.append(",");
            }
            json.append("{");
            json.append("\"id\":").append(resultado.getInt("ID_Cliente")).append(",");

            // CORRECCIÓN: Se agregaron las comillas \" al inicio y al final de los textos
            json.append("\"nombre\":\"").append(escapar Json(resultado.getString("Nombre"))).append("\",");
```

API para ver los clientes ya registrados en la base de datos y para registrar un nuevo cliente

```
                json.append("\"nombre\":\"").append(escaparJson(resultado.getString("Nombre"))).append("\",");
                json.append("\"apellido\":\"").append(escaparJson(resultado.getString("Apellido"))).append("\",");
                json.append("\"dni\":\"").append(escaparJson(resultado.getString("DNI"))).append("\",");
                json.append("\"email\":\"").append(escaparJson(resultado.getString("email"))).append("\",");
                json.append("\"telefono\":\"").append(escaparJson(resultado.getString("telefono"))).append("\",");
                json.append("\"direccion\":\"").append(escaparJson(resultado.getString("direccion"))).append("\",");
                json.append("\"postal\":\"").append(escaparJson(resultado.getString("cod_postal"))).append("\"");
                json.append("}");
                primero = false;
            }
            json.append("]");

            // CORRECCIÓN EXTRA: Asegúrate de que enviarRespuesta configure el Content-Type como "application/json"
            intercambio.getResponseHeaders().set("Content-Type", "application/json; charset=UTF-8");

            enviarRespuesta(intercambio, 200, json.toString());
            System.out.println("Llegó hasta aquí con éxito. JSON enviado.");

        } catch (SQLException e) {
            e.printStackTrace();
            enviarRespuesta(intercambio, 500, "{\"error\":\"Error al consultar la base de datos\"}");
        }
    }
            });
             servidor.createContext("/api/Clientes/crear", new HttpHandler() {

                @Override
                public void handle(HttpExchange intercambio) throws IOException {

                    if (!intercambio.getRequestMethod().equals("POST")) {
                        enviarRespuesta(intercambio, 405,
                                "{\"error\":\"Metodo no permitido\"}");
                        return;
                    }
```

```
                    // Leer los datos que llegan desde el formulario.
                    java.io.InputStream entrada = intercambio.getRequestBody();
                    java.io.ByteArrayOutputStream buffer =
                            new java.io.ByteArrayOutputStream();

                    int dato;
                    while ((dato = entrada.read()) != -1) {
                        buffer.write(dato);
                    }

                    String cuerpo = new String(
                            buffer.toByteArray(),
                            StandardCharsets.UTF_8
                    );

                    // Separar los datos recibidos.
                    java.util.Map<String, String> datos =
                            new java.util.HashMap<String, String>();

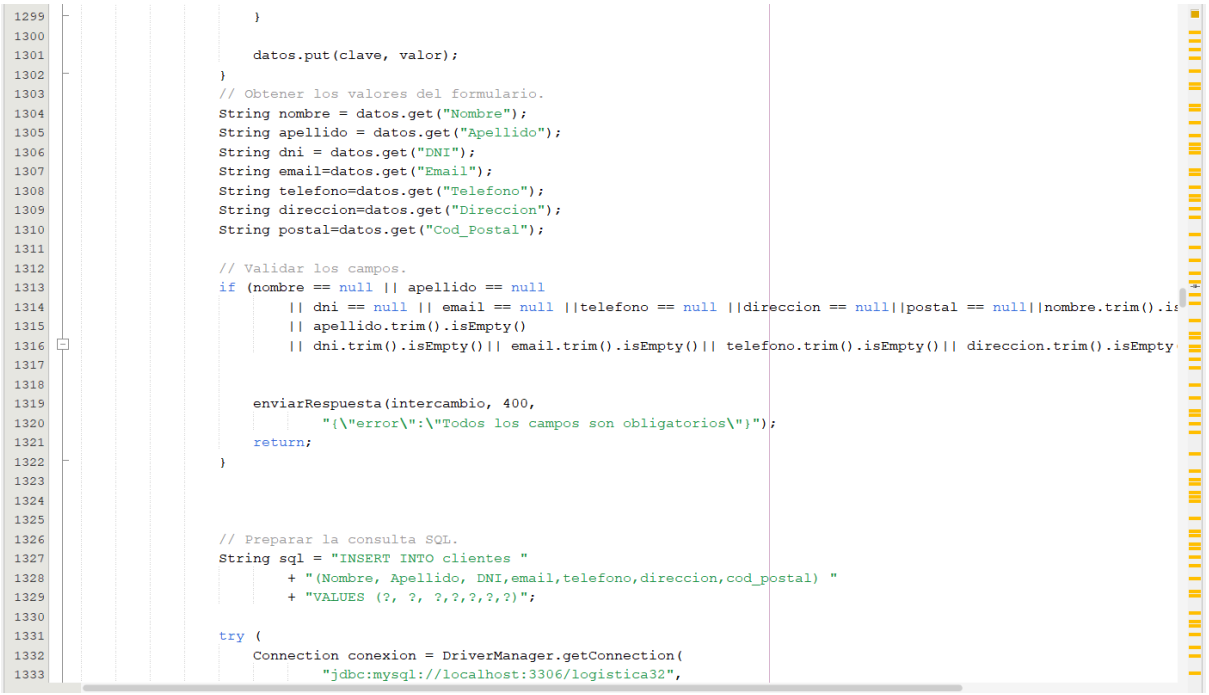
                    String[] parametros = cuerpo.split("&");

                    for (String parametro : parametros) {
                        String[] partes = parametro.split("=", 2);

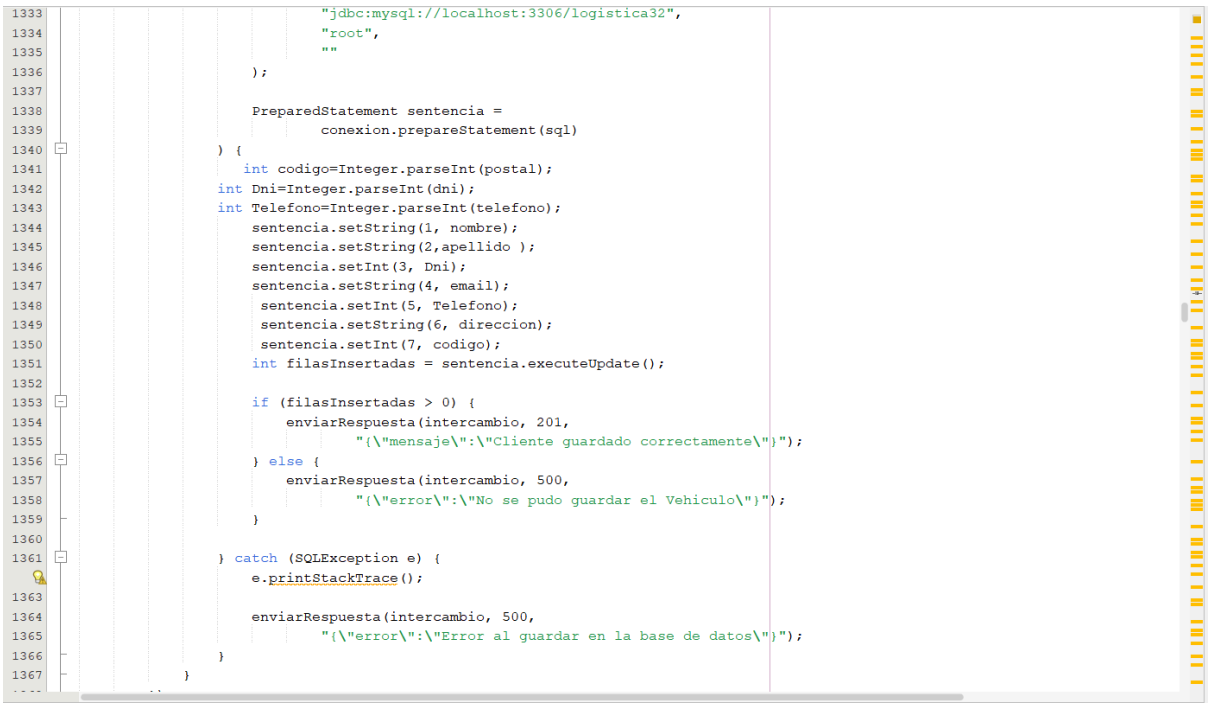
                        String clave = java.net.URLDecoder.decode(
                                partes[0], "UTF-8"
                        );

                        String valor = "";

                        if (partes.length > 1) {
                            valor = java.net.URLDecoder.decode(
                                    partes[1], "UTF-8"
                            );
```



```
1299                    }
1300
1301                    datos.put(clave, valor);
1302                }
1303                // Obtener los valores del formulario.
1304                String nombre = datos.get("Nombre");
1305                String apellido = datos.get("Apellido");
1306                String dni = datos.get("DNI");
1307                String email=datos.get("Email");
1308                String telefono=datos.get("Telefono");
1309                String direccion=datos.get("Direccion");
1310                String postal=datos.get("Cod_Postal");
1311
1312                // Validar los campos.
1313                if (nombre == null || apellido == null
1314                        || dni == null || email == null ||telefono == null ||direccion == null||postal == null||nombre.trim().is
1315                        || apellido.trim().isEmpty()
1316                        || dni.trim().isEmpty()|| email.trim().isEmpty()|| telefono.trim().isEmpty()|| direccion.trim().isEmpty
1317
1318
1319                    enviarRespuesta(intercambio, 400,
1320                            "{\"error\":\"Todos los campos son obligatorios\"}");
1321                    return;
1322                }
1323
1324
1325
1326                // Preparar la consulta SQL.
1327                String sql = "INSERT INTO clientes "
1328                        + "(Nombre, Apellido, DNI,email,telefono,direccion,cod_postal) "
1329                        + "VALUES (?, ?, ?,?,?,?,?)";
1330
1331                try (
1332                    Connection conexion = DriverManager.getConnection(
1333                            "jdbc:mysql://localhost:3306/logistica32",
```



```
1333                            "jdbc:mysql://localhost:3306/logistica32",
1334                            "root",
1335                            ""
1336                    );
1337
1338                    PreparedStatement sentencia =
1339                            conexion.prepareStatement(sql)
1340                ) {
1341                   int codigo=Integer.parseInt(postal);
1342                int Dni=Integer.parseInt(dni);
1343                int Telefono=Integer.parseInt(telefono);
1344                    sentencia.setString(1, nombre);
1345                    sentencia.setString(2,apellido );
1346                    sentencia.setInt(3, Dni);
1347                    sentencia.setString(4, email);
1348                     sentencia.setInt(5, Telefono);
1349                     sentencia.setString(6, direccion);
1350                     sentencia.setInt(7, codigo);
1351                    int filasInsertadas = sentencia.executeUpdate();
1352
1353                    if (filasInsertadas > 0) {
1354                        enviarRespuesta(intercambio, 201,
1355                                "{\"mensaje\":\"Cliente guardado correctamente\"}");
1356                    } else {
1357                        enviarRespuesta(intercambio, 500,
1358                                "{\"error\":\"No se pudo guardar el Vehiculo\"}");
1359                    }
1360
1361                } catch (SQLException e) {
1362                    e.printStackTrace();
1363
1364                    enviarRespuesta(intercambio, 500,
1365                            "{\"error\":\"Error al guardar en la base de datos\"}");
1366                }
1367            }
```

API para editar un cliente

```
1368        });
            servidor.createContext("/api/Clientes/editar", new HttpHandler() {
1370
1371            @Override
1372            public void handle(HttpExchange intercambio) throws IOException {
1373
1374                if (!intercambio.getRequestMethod().equals("POST")) {
1375                    enviarRespuesta(intercambio, 405,
1376                            "{\"error\":\"Metodo no permitido\"}");
1377                    return;
1378                }
1379
1380                java.util.Map<String, String> datos = leerParametros(intercambio);
1381
1382                try {
1383                    int ID_Cliente = Integer.parseInt(datos.get("id"));
1384                    String nombre = datos.get("Nombre");
1385                    String Apellido = datos.get("Apellido");
1386                    int DNI= Integer.parseInt(datos.get("DNI"));
1387                    String email = datos.get("email");
1388                    int telefono = Integer.parseInt(datos.get("telefono"));
1389                    String direccion = datos.get("direccion");
1390                    int cod_postal = Integer.parseInt(datos.get("cod_postal"));
1391
1392
1393                    if (telefono <= 0) {
1394
1395                        enviarRespuesta(intercambio, 400,
1396                                "{\"error\":\"Revisa los datos ingresados\"}");
1397                        return;
1398                    }
1399
1400                    String sql = "UPDATE Clientes SET Nombre = ?, Apellido = ?, "
1401                        + "DNI = ?, email = ?, "
1402                        + "telefono = ?, direccion = ?, "
1402                        + "telefono = ?, direccion = ?, "
1403                        + "cod_postal = ? "
1404                        + "WHERE ID_Cliente = ?"
1405                    ;
1406
1407                    try (
1408                        Connection conexion = DriverManager.getConnection(
1409                                "jdbc:mysql://localhost:3306/logistica32", "root", "");
1410                        PreparedStatement sentencia = conexion.prepareStatement(sql)
1411                    ) {
1412                        sentencia.setString(1, nombre);
1413                        sentencia.setString(2, Apellido);
1414                        sentencia.setInt(3, DNI);
1415                        sentencia.setString(4, email);
1416                        sentencia.setInt(5, telefono);
1417                        sentencia.setString(6, direccion);
1418                        sentencia.setInt(7, cod_postal);
1419                        sentencia.setInt(8, ID_Cliente);
1420
1421                        int filas = sentencia.executeUpdate();
1422
1423                        if (filas > 0) {
1424                            enviarRespuesta(intercambio, 200,
1425                                    "{\"mensaje\":\"Cliente actualizado correctamente\"}");
1426                        } else {
1427                            enviarRespuesta(intercambio, 404,
1428                                    "{\"error\":\"No se encontro el cliente\"}");
1429                        }
1430                    }
1431
1432                } catch (NumberFormatException e) {
1433                    enviarRespuesta(intercambio, 400,
1434                            "{\"error\":\"telefono y DNI deben ser numero validos\"}");
1435                } catch (SQLException e) {
                        e.printStackTrace();
```

API para eliminar Clientes



```
                    e.printStackTrace();
                    enviarRespuesta(intercambio, 500,
                            "{\"error\":\"Error al actualizar la base de datos\"}");
                } catch (Exception e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"Faltan datos o son invalidos\"}");
                }
            }
        });
        servidor.createContext("/api/Clientes/eliminar", new HttpHandler() {

            @Override
            public void handle(HttpExchange intercambio) throws IOException {

                if (!intercambio.getRequestMethod().equals("POST")) {
                    enviarRespuesta(intercambio, 405,
                            "{\"error\":\"Metodo no permitido\"}");
                    return;
                }

                java.util.Map<String, String> datos = leerParametros(intercambio);

                try {
                    int id_empresa = Integer.parseInt(datos.get("id"));

                    if (id_empresa <= 0) {
                        enviarRespuesta(intercambio, 400,
                                "{\"error\":\"ID no valido\"}");
                        return;
                    }

                    String sql = "DELETE FROM clientes WHERE ID_Cliente = ?";

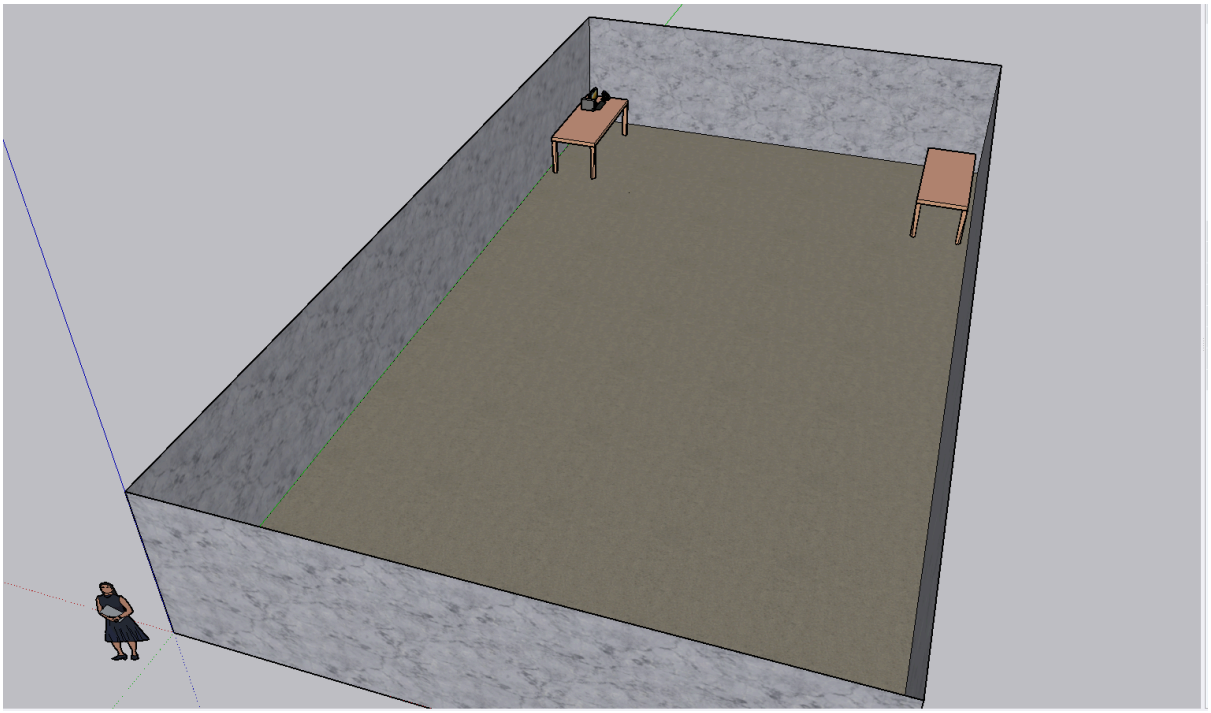
                    try (
                        Connection conexion = DriverManager.getConnection(
                                "jdbc:mysql://localhost:3306/logistica32", "root", "");
                        PreparedStatement sentencia = conexion.prepareStatement(sql)
                    ) {
                        sentencia.setInt(1, id_empresa);

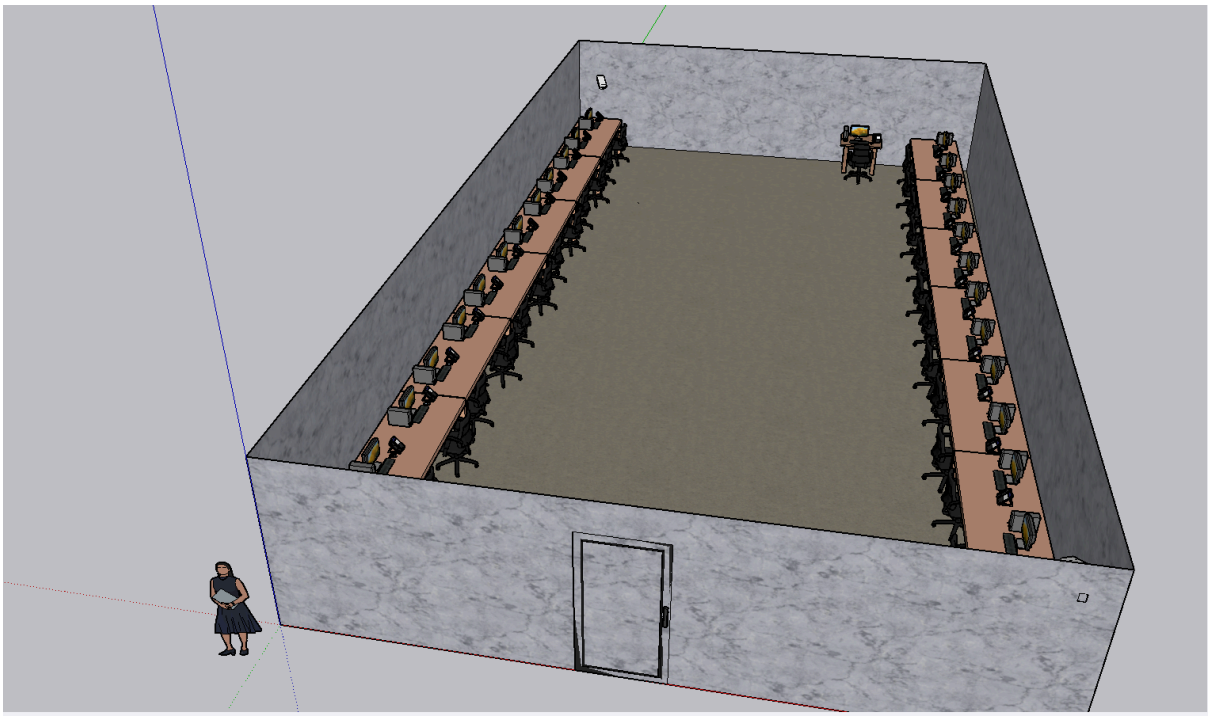
                        int filas = sentencia.executeUpdate();

                        if (filas > 0) {
                            enviarRespuesta(intercambio, 200,
                                    "{\"mensaje\":\"Clienta eliminada correctamente\"}");
                        } else {
                            enviarRespuesta(intercambio, 404,
                                    "{\"error\":\"No se encontro la persona\"}");
                        }
                    }

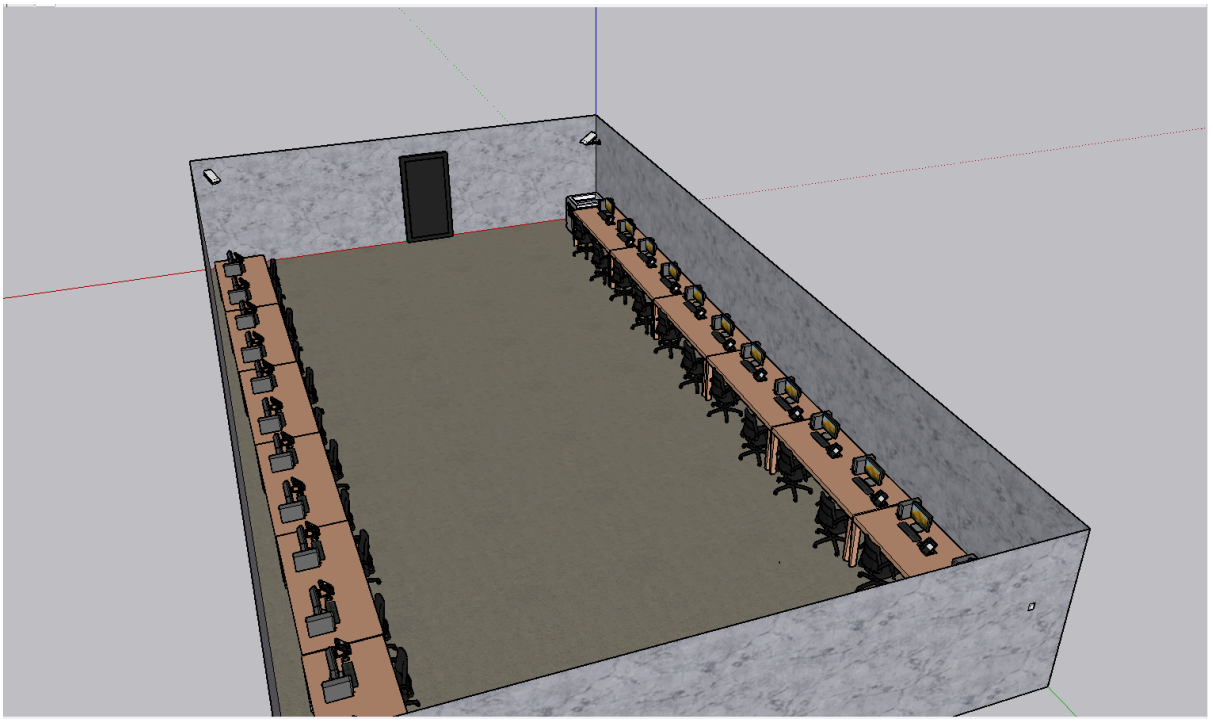
                } catch (NumberFormatException e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"El ID debe ser numerico\"}");
                } catch (SQLException e) {
                    e.printStackTrace();
                    enviarRespuesta(intercambio, 500,
                            "{\"error\":\"Error al eliminar de la base de datos\"}");
                } catch (Exception e) {
                    enviarRespuesta(intercambio, 400,
                            "{\"error\":\"No se pudo procesar la solicitud\"}");
                }
            }
        });
```

5/11 empezé a hacer el piso 7, en el cual requeria una habitacion de 18.6m de largo x 10.2m de ancho, con 2.8m de alto. hice la habitacion en sketch up y luego busque en las eccion de 3d warehouse de la aplicación una computadora, un telefono, una mesa, una camara y una impresora de red.





como se puede apreciar. son 2 filas de mesas. 6 mesas en cada fila. Cada mesa tiene 2 computadoras junto con 2 teléfonos ip y el respectivo asiento. Como me faltó 1 computadora y 1 teléfono más, descargué una mesa pequeña para ponerla al lado de una de las filas y ahí agregué la pc y teléfono faltante. en 1 de las filas al lado de la puerta se encuentra la impresora de red. también habiendo 3 cámaras donde asigne 2 en las esquinas donde se encuentra la puerta y la otra la coloqué en el fondo del lado izquierdo



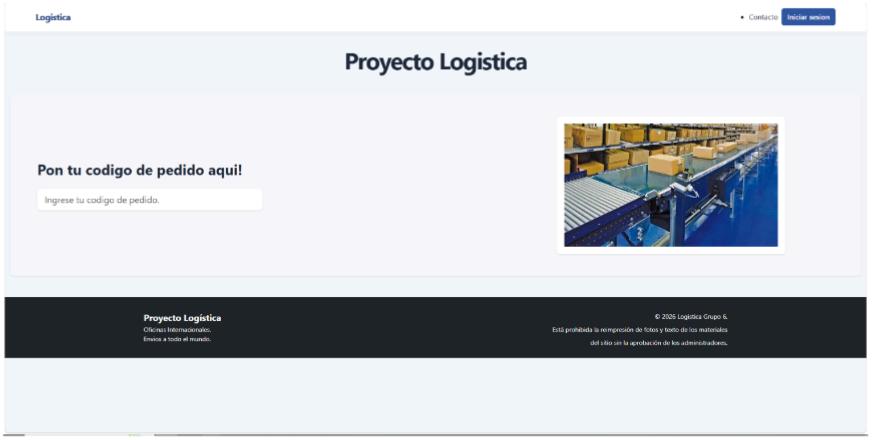
(lo de la puerta lo soluciono despues)

6/11
Hice una de las cosas que faltaba por terminar que era hacer el manual de usuario para las páginas de cliente y administrador. Al momento de hacerlo, estuve viendo imperfecciones en el CSS que fueron corregidos y un error en la página del administrador. específicamente en el ver clientes. ya que al clickear no encontraba el archivo, no hubo problema en solucionarlo ya que solo cambiamos el nombre del link de “mostrar cliente.html” a “VerClientes:html” que así es como se llama la pagina para ver los clientes registrados

MANUAL DE USUARIO:

allí explique el uso de cada pagina. tanto del lado del cliente como el del administrador. las explicaciones fueron bastante cortas y claras de entender.
EJ:

1) Página Main (cliente)

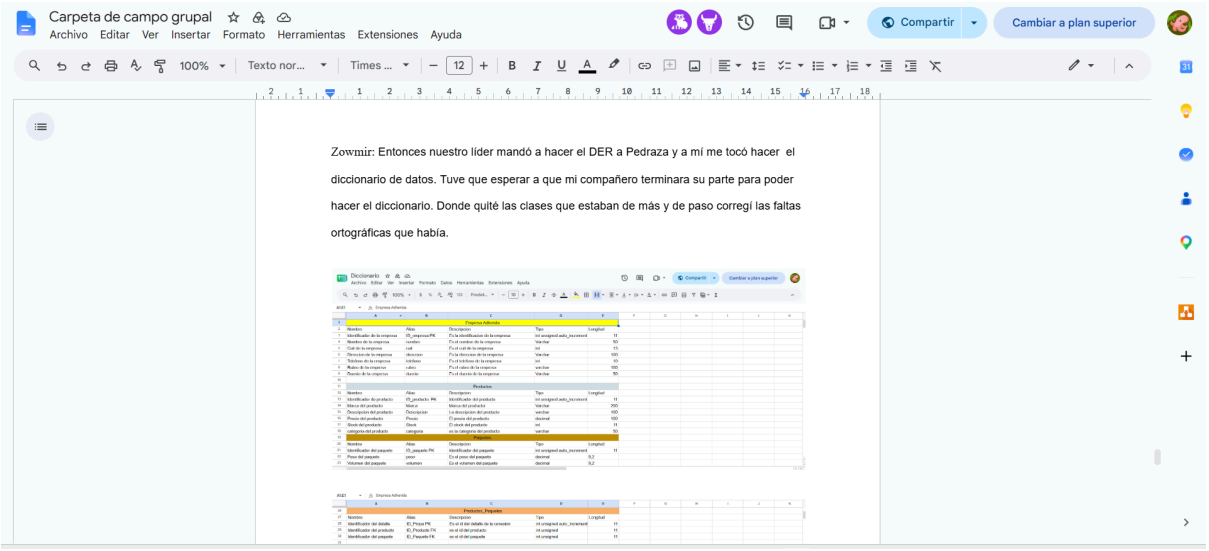


en la página main el cliente tiene las opciones de poner el código de su pedido para ver los datos (Destino,Origen,Estado del pedido, etc.), tiene el botón de contactos para que se pueda contactar con el equipo de soporte por si hay algún problema y tiene el botón de iniciar sesion para que pueda abrir su cuenta(o crear una) en la página.

explicacion de como se usa la pagina main del cliente

Además, a último momento me encargué de terminar la carpeta de campo grupal que había comenzado mi compañero Santiago Pedraza. El objetivo de este trabajo era reunir en un solo documento PDF todas las actividades y tareas que fuimos realizando durante el desarrollo del proyecto, para dejar registrado el trabajo de cada integrante y el avance que tuvimos como grupo.

Para completarla, tuve que recopilar y organizar la información y los trabajos realizados, siguiendo como referencia el ejemplo que se muestra a continuación. De esta manera, buscamos que quedara un registro ordenado de todo el proceso del proyecto.



Carpeta de campo grupal

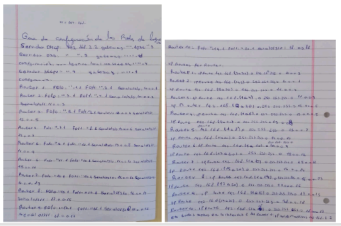
Archivo Editar Ver Insertar Formato Herramientas Extensiones Ayuda

Compartir | Cambiar a plan superior

100% | Texto nor... | Times ... | 12

Saragusti: además de la revisión yo termine de configurar y conectar las últimas 10 redes que faltaban con dhcp,dns y servidor http funcional, que igual no tuve que modificar los datos de los routers ya que previamente los había dejado preparados para la extensión de la red.

También hice una hoja escrita en la que deje la estructura plasmada de la configuración de los equipos para que mis compañeros pudieran entender la estructura y como estaba armada la red, ya que una explicación oral no servirá tanto como el código plasmado en texto de una manera sencilla para fácil deducción de los ideales.



30 de 37